Escola de Engenharia Elétrica, Mecânica e de Computação

CERISE – Centro de Excelência em Redes Inteligentes Sem Fio e Serviços Avançados

Projeto de Implantação do CERISE-Centro de Excelência em Redes Inteligentes Sem fio e Serviços Avançados com Alcance Nacional e Internacional

Roteiro de Implementação

Título do Plano de Trabalho Individual: Pesquisa, Desenvolvimento e Inovação em Gêmeos Digitais, Tecnologias Imersivas, Robótica e Navegação Autônoma no escopo do CERISE/EMC/UFG

Bolsista: Eduardo Martins de Carvalho Caixeta

Grupo / Eixo Temático: Gêmeos Digitais / Navegação Autônoma / Realidade Aumentada

Coordenadores dos Grupos / Eixos Temáticos: Prof. Dr. Alisson Assis Cardoso (Navegação Autônoma e Gêmeos Digitais); Prof. Dr. Carlos Galvão Pinheiro Junior (Realidade Aumentada)

Versão: 26/10/2026

1. APRESENTAÇÃO

Este documento é o Roteiro de Implementação do plano de trabalho individual "Pesquisa, Desenvolvimento e Inovação em Gêmeos Digitais, Tecnologias Imersivas, Robótica e Navegação Autônoma no escopo do CERISE/EMC/UFG", executado em 2026. Ele reúne as instruções para que outra pessoa reproduza, a partir do código e dos dados publicados, os protótipos, as implementações e as demonstrações do plano: a geração de modelos 3D a partir de fotografias e o servidor que atende vários usuários; o aplicativo CERISE AR 1.1.0; o dataset sintético, o modelo digital e o gêmeo digital do braço robótico servo6dof; o modelo de estimação de pose do braço; o protótipo de comunicação LoRa; e a segmentação de área, a detecção e a contagem de pessoas com drones, inclusive em simulação 3D.

O público-alvo é um aluno ou pesquisador que nunca teve contato com o projeto. Supõe-se familiaridade com o terminal do Linux, o Git, o Docker e o Python; as seções do aplicativo e do LoRa pedem também noções de Android Studio, Xcode e PlatformIO. Cada seção indica o hardware necessário, o tempo estimado e o que deve aparecer ao final de cada etapa.

1.1. Organização do documento

A Seção 2 explica como obter acesso aos repositórios, instalar as ferramentas comuns a todas as frentes (Git, GitHub CLI, Docker e NVIDIA Container Toolkit), baixar os arquivos grandes das Releases e conferir as somas SHA-256. As Seções 3 a 8 trazem um roteiro por frente de trabalho, e a Seção 9, os contatos.

Todas as seções de frente seguem a mesma estrutura: o que será reproduzido; requisitos de hardware, software, contas, disco e tempo; obtenção do código e dos dados; as etapas, com os comandos na ordem de execução; a verificação do resultado, com os valores de referência medidos no projeto e o arquivo do repositório em que cada um está registrado; os problemas comuns e as soluções; e as referências do repositório.

Tabela 1. Seções do roteiro, atividades do plano de trabalho e dependências entre as frentes

Seção	Frente	Atividades do plano	Depende de
3	Geração 3D e servidor de geração	Objetivo 1, Atividades 1, 3 e 4 e lado servidor da 2; Objetivo 2, Atividade 5	nenhuma
4	Aplicativo CERISE AR	Objetivo 1, Atividade 2; Objetivo 2, Atividade 6	servidor da Seção 3 ou o servidor simulado do próprio repositório
5	Dataset, modelo digital e gêmeo digital do braço	Objetivo 3, Atividade 7; Objetivo 4, Atividades 11 e 12	modo com o modelo de pose: Seção 6 ou o modelo das Releases
6	Estimação de pose do braço	Objetivo 3, Atividades 8 e 9; interface com a Atividade 12	dataset da Seção 5 (Releases)
7	Protótipo de comunicação LoRa	Objetivo 3, Atividade 10; Objetivo 5, itens 3 e 4 (projeto)	teste de compatibilidade opcional com a Seção 8
8	Drones: segmentação, detecção, contagem e simulação	Objetivo 5, Atividade 13, item 1 e algoritmos de colaboração	nenhuma

As frentes podem ser reproduzidas de forma independente. Quando houver dependência, a ordem sugerida é 3 → 4 (o aplicativo usa o servidor) e 5 → 6 → 5 (o dataset alimenta o treino da pose, e o modelo de pose alimenta o gêmeo digital). Em todos os casos, as Releases trazem os artefatos prontos, como o dataset e o modelo final de pose, o que permite começar por qualquer seção.

1.2. Relação com o relatório técnico-científico

Este roteiro complementa o Relatório Técnico-Científico CERISE 2026 do mesmo plano de trabalho. O relatório descreve os objetivos, os métodos, os resultados e a comparação com o plano; o roteiro descreve como chegar aos mesmos resultados. Os valores de referência das seções de verificação são os números medidos e apresentados no relatório, com a indicação do arquivo de resultados (JSON, CSV ou log) em que cada um está registrado no repositório. O conteúdo de cada frente também está no arquivo `ROTEIRO.md` do repositório correspondente, com links para os arquivos citados.

1.3. Convenções

- Os comandos aparecem em blocos de fundo cinza, em fonte monoespaçada. Salvo indicação, são comandos do Bash num terminal Linux ou macOS; os blocos com caminhos do tipo `C:\...` ou com `$env:` são do PowerShell, no Windows.
- Uma barra invertida (`\`) no fim de uma linha do Bash, ou um acento grave no fim de uma linha do PowerShell, indica que o comando continua na linha seguinte; os comandos longos foram quebrados

assim para caber na página. Copie o bloco inteiro e confira, ao colar a partir do PDF, se nenhuma linha foi unida ou partida.

- O texto entre < e > deve ser substituído pelo valor do leitor, por exemplo `<usuario>@<maquina-gpu>`, `<id>` ou `<seu-dominio>`. O símbolo `~` e a variável `$HOME` indicam a pasta pessoal do usuário.
- Os caminhos relativos partem da raiz do repositório, salvo quando o próprio passo muda de pasta com `cd`. Caminhos como `/workspace`, `/w`, `/ws`, `/dados` e `/runs` existem só dentro dos contêineres: são os pontos de montagem definidos nos comandos `docker run` e nos scripts.
- Os comandos Docker usam `sudo`, como os scripts dos repositórios. Mesmo que o seu usuário pertença ao grupo `docker`, mantenha o `sudo` disponível, porque vários scripts o chamam internamente.
- No texto, os números decimais usam vírgula; nos comandos e nos arquivos, ponto.
- A indicação "não testado" marca adaptações (por exemplo, para x86_64) que fazem sentido pelo código, mas não foram executadas no projeto.
- Os tempos foram medidos numa estação compartilhada com outros serviços e devem ser lidos como limites superiores. Contagens, métricas de qualidade e tamanhos de arquivo devem coincidir de perto quando se usam os mesmos dados e modelos.

2. ACESSO AOS REPOSITÓRIOS E REQUISITOS GERAIS

2.1. Organização cerise-ufg e pedido de acesso

Todo o código, os resultados medidos e as evidências estão na organização cerise-ufg do GitHub, em sete repositórios privados. Para obter acesso:

1. Crie uma conta no GitHub, se ainda não tiver uma.

2. Solicite acesso ao bolsista (Seção 9), informando o seu nome de usuário no GitHub e os repositórios de interesse.

3. Aceite o convite recebido por e-mail ou na página da organização. Sem o convite, os comandos `git clone`, `gh repo clone` e `gh release download` respondem que o repositório não existe.

Os dados e os pesos de terceiros (Google Scanned Objects, pesos do Hunyuan3D-2 e do Depth Anything V2, VisDrone, ativos do Isaac Sim, entre outros) não são redistribuídos: cada seção indica de onde baixá-los, e a licença de cada um deve ser lida e aceita pelo próprio leitor.

Tabela 2. Repositórios da organização cerise-ufg e seção correspondente deste roteiro

Repositório	O que contém	Roteiro
`cerise-2026-gen3d`	Comparação de modelos de geração 3D, pipeline do Hunyuan3D-2 Turbo com três *presets*, validação dos GLBs e servidor de geração 3D com fila (pasta `server`)	Seção 3 e `ROTEIRO.md`
`cerise-ar` (branch `feat/2026-gen3d-playstore`)	Aplicativo CERISE AR 1.1.0 (Expo/React Native), servidor simulado da API, servidor de atualizações OTA e pacote da Play Store	Seção 4 e `ROTEIRO.md`
`cerise-2026-servo6dof-gemeo-digital`	Gerador do dataset no Gazebo, ficha e amostra do dataset, modelo digital (URDF, cinemática direta e MoveIt 2) e gêmeo digital (servidor web, front-end three.js e nó ROS 2)	Seção 5 e `ROTEIRO.md`
`cerise-2026-pose-servo6dof`	Porte do modelo de pose de 2025, conversão do dataset, treino, avaliação e módulo de inferência	Seção 6 e `ROTEIRO.md`
`cerise-2026-lora-prototipo`	Protocolo `cerise_proto`, firmware do nó e do gateway, testes, ferramentas de análise, dashboard e projeto do módulo de comunicação	Seção 7 e `ROTEIRO.md`
`cerise-2026-drones-segmentacao`	Segmentação de área e missões, colaboração entre drones, detector de pessoas, contagem em vídeos reais e simulação no Isaac Sim	Seção 8 e `ROTEIRO.md`
`cerise-2026-relatorio`	Relatório técnico-científico de 2026 (texto-fonte, .docx e .pdf), gerador dos documentos e anexos	não se aplica (documentação)

2.2. Máquinas e requisitos de hardware

O projeto usou três máquinas. A tabela abaixo as descreve como requisitos; os detalhes de memória, disco e tempo de cada etapa estão na seção de cada frente.

Tabela 3. Máquinas usadas no projeto e alternativas para a reprodução

Máquina usada	Características	Seções	Alternativa
NVIDIA DGX Spark	estação de GPU: GB10 (Blackwell, sm_121), CPU aarch64, 128 GB de memória unificada, Ubuntu 24.04, Docker e NVIDIA Container Toolkit	3, 5, 6 e 8 (detecção, vídeos e simulação)	Linux x86_64 com GPU NVIDIA e driver com suporte a CUDA 13, com as adaptações indicadas em cada seção (não testado)
Mac mini M4	Apple Silicon (M4), macOS 26 e Xcode 26	4 (iOS)	outro Mac com Apple Silicon e Xcode 26
Notebook Windows 11	x86_64; Android SDK e emulador, JDK 17, Node.js, PlatformIO e Python 3.11	4 (Android), 7 e 8 (segmentação)	Linux ou macOS, não testados para o Android e o LoRa

A GPU necessária varia por frente: o treino de pose pede pelo menos 12 GB; o servidor de geração 3D, mais de 24 GB; a simulação no Isaac Sim, uma GPU RTX. Na estação usada, CPU e GPU compartilham a mesma memória; numa GPU dedicada, confira separadamente a memória de vídeo e a memória RAM disponível para os contêineres.

2.3. Git e GitHub CLI

1. Instale o Git e a GitHub CLI (`gh`).

```
sudo apt update && sudo apt install -y git gh      # Ubuntu 24.04
brew install git gh                                # macOS, com o Homebrew

winget install --id Git.Git -e
winget install --id GitHub.cli -e
```

2. Autentique-se com a sua própria conta. Escolha GitHub.com, o protocolo HTTPS e a opção de autenticar o Git com as credenciais do GitHub, e conclua o login no navegador. O projeto não fornece tokens nem senhas; não use credenciais de outra pessoa.

```
gh auth login
gh auth status
```

3. Clone cada repositório com `gh repo clone cerise-ufg/<repositorio> <pasta>`. Vários scripts supõem pastas fixas, como `~/cerise2026/gen3d`, `~/cerise2026/gemeo/repo`, `~/cerise2026/pose`, `~/cerise2026/drones` e, no Mac, `~/CeriseAR`; use sempre a pasta indicada na seção da frente.

2.4. Releases e verificação SHA-256

Os arquivos grandes (dataset, pesos treinados, saídas das simulações e pacotes do aplicativo) não estão no histórico do Git, mas anexados às Releases de cada repositório. O arquivo `RELEASES.md` de cada um lista os anexos, os tamanhos, as somas SHA-256 e onde extrair cada pacote; em geral, os `.tar` são extraídos na raiz do repositório, porque já trazem os caminhos relativos.

Tabela 4. Releases usadas nos roteiros e arquivos com as somas SHA-256

Repositório	Release (tag)	Somas SHA-256
`cerise-2026-gen3d`	`dados-2026-10-26`	`SHA256SUMS`, anexado à Release
`cerise-ar`	`v1.1.0` e `v1.0.0`	tabela do `RELEASES.md`
`cerise-2026-servo6dof-gemeo-digital`	`dados-2026-10-26`	`SHA256SUMS.txt` e manifesto por arquivo do dataset
`cerise-2026-pose-servo6dof`	`artefatos-2026-10-26`	`SHA256SUMS.txt`
`cerise-2026-lora-prototipo`	`v1.0-prototipo`	`RELEASES.md` e `SHA256SUMS.txt` dentro do zip
`cerise-2026-drones-segmentacao`	`dados-2026-10-26`	`SHA256SUMS.txt`
`cerise-2026-relatorio`	`v2026.10.26`	`SHA256SUMS.txt`

1. Liste as Releases e baixe os anexos. A opção `-p` restringe o download aos arquivos que casam com o padrão e pode ser repetida; sem ela, o `gh` baixa todos os anexos.

```
gh release list -R cerise-ufg/<repositorio>
gh release download <tag> -R cerise-ufg/<repositorio> -D <pasta> -p '<padrao>'
```

2. Confira as somas antes de extrair. Se baixou só parte dos anexos, acrescente `--ignore-missing`.

```
cd <pasta> && sha256sum -c SHA256SUMS.txt          # Linux
shasum -a 256 -c SHA256SUMS.txt                     # macOS

Get-FileHash .\<arquivo> -Algorithm SHA256          # compare com o RELEASES.md
```

Cada linha deve terminar em `OK`. Uma soma divergente indica download incompleto: apague o arquivo e baixe de novo.

2.5. Docker e NVIDIA Container Toolkit

As Seções 3, 5, 6 e 8 rodam em contêineres Docker com acesso à GPU. Instale, na máquina de GPU:

- o driver NVIDIA com suporte a CUDA 13 (o projeto usou a série 580);
- o Docker Engine com o plugin Compose v2, conforme a documentação oficial do Docker para o Ubuntu;
- o NVIDIA Container Toolkit, conforme o guia de instalação da NVIDIA.

Em seguida, registre o runtime da NVIDIA no Docker e teste o acesso à GPU com a imagem base usada pelas frentes de geração 3D, pose e detecção (o primeiro download leva alguns minutos):

```
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker
sudo docker run --rm --gpus all nvcr.io/nvidia/pytorch:26.05-py3 nvidia-smi
```

O `nvidia-smi` deve listar a sua GPU. Se o contêiner não enxergar a GPU, revise a instalação do toolkit antes de seguir para as seções das frentes.

2.6. Observação sobre aarch64 e x86_64

Todo o trabalho com GPU foi executado em aarch64, na GPU GB10 (sm_121), com o CUDA 13.2 da imagem NGC 26.05. A maior parte do código não depende da arquitetura: Python, imagens NGC do PyTorch e imagens do ROS 2 também são publicadas para x86_64. Alguns pontos, porém, estão fixados para arm64 ou sm_121 e precisam ser adaptados numa máquina x86_64 com GPU NVIDIA comum. Nenhuma dessas adaptações foi testada:

- na geração 3D (Seção 3), o Node.js para `linux-arm64` no `docker/Dockerfile` e `TORCH_CUDA_ARCH_LIST="12.1"` no Dockerfile e nos scripts `docker/install_*.sh`, que devem receber a arquitetura da sua GPU;
- no gêmeo digital (Seção 5), o repositório apt da OSRF com `arch=arm64` no `docker/Dockerfile.spark`;
- na detecção de pessoas (Seção 8), o FP16 gerou NaN no GB10 e os scripts usam FP32; noutra GPU, rode o teste de precisão numérica antes de ativar o FP16;
- no treino de pose (Seção 6), GPUs sem bfloat16 (anteriores à arquitetura Ampere) exigem `--sem_amp`;
- na geração 3D, TRELLIS e TRELLIS.2 ficaram fora do benchmark por falta de pacotes para aarch64; em x86_64, não foram avaliados.

O aplicativo (Seção 4) e o protótipo LoRa (Seção 7) não dependem da estação de GPU: rodam num computador Windows x86_64 e, no caso do iOS, num Mac com Apple Silicon.

3. GERAÇÃO 3D A PARTIR DE FOTOGRAFIA E SERVIDOR DE GERAÇÃO

3.1. O que será reproduzido

Este roteiro mostra como montar, a partir do repositório `cerise-2026-gen3d`, o pipeline que transforma a fotografia de um objeto em um modelo 3D texturizado (GLB) para o celular e o servidor que atende vários usuários com fila e um worker de GPU. O modelo é o Hunyuan3D-2 Turbo com FlashVDM, com três *presets* (rápido, padrão e alto). Os caminhos citados são relativos à raiz do repositório.

Ao final, você terá:

- a imagem Docker `cerise26_gen3d:latest` e a classe `Gen3DPipeline`, que gera um GLB com maior dimensão de 0,30 m, apoiado em y = 0, com textura JPEG e sem Draco, meshopt ou KTX2 (Objetivo 1, Atividade 3);

- as métricas do benchmark A1 (Chamfer, F-score, CLIP, tempo por etapa e pico de GPU) recalculadas em 13 objetos do Google Scanned Objects (Objetivo 1, Atividade 1);
- a validação A4: 58 testes pytest e o carregamento no `<model-viewer>` e no three.js (Objetivo 1, Atividade 4);
- o servidor de geração 3D (API FastAPI, fila SQLite e worker de GPU em Docker Compose), testado com curl, com o endpoint legado de 2025, com o aplicativo e com a suíte de carga (Objetivo 2, Atividade 5, e lado servidor da Atividade 2 do Objetivo 1).

3.2. Requisitos

Tabela 5. Requisitos para reproduzir a geração 3D e o servidor

Item	Usado no projeto	Para reproduzir
GPU	NVIDIA DGX Spark: GB10 Blackwell (sm_121), 128 GB de memória unificada, teto de 24 GB para esta frente	GPU NVIDIA com mais de 24 GB; picos medidos de 14,5 GB (pipeline) e, no worker, 18,9 GB alocados (22,9 GB no `nvidia-smi`); com 24 GB, suba o worker sem o *preset* alto (ver Problemas comuns)
Sistema	Ubuntu 24.04, CPU aarch64	Linux; x86_64 com as adaptações indicadas (não testado)
Contêineres	Docker com Compose v2 e NVIDIA Container Toolkit; base `nvcr.io/nvidia/pytorch:26.05-py3` (PyTorch 2.12, CUDA 13.2, Python 3.12)	as mesmas versões e driver compatível com CUDA 13
Disco	pesos ~70 GB (só o pipeline: ~37 GB), imagem ~38 GB, Releases ~1,9 GB	150 GB livres
Cliente	notebook Windows com Python 3 e `requests`; Android SDK com emulador	qualquer máquina com `curl` e `ssh`
Contas	GitHub com acesso à organização `cerise-ufg` e GitHub CLI (`gh`)	as mesmas; conta no Hugging Face é opcional

Tempo estimado: 12 min para construir a imagem; o download dos pesos depende da rede; 3 min para carregar e aquecer o modelo; cerca de 20 min para o Hunyuan3D-2 Turbo no benchmark; 6 min para os testes A4; 40 min para a suíte de carga. Os repositórios da organização são privados: solicite acesso ao bolsista. Os pesos do Hunyuan3D-2 seguem a *Tencent Hunyuan 3D Community License*, que não vale na União Europeia, no Reino Unido e na Coreia do Sul.

3.3. Obtenção do código e dos dados

1. Autentique a GitHub CLI (escolha HTTPS e aceite autenticar o Git com as suas credenciais do GitHub) e clone o repositório exatamente em `~/cerise2026/gen3d`, caminho usado por vários scripts.

```
gh auth login
gh repo clone cerise-ufg/cerise-2026-gen3d ~/cerise2026/gen3d
mkdir -p ~/cerise2026/hf_cache
```

2. Clone o Hunyuan3D-2 e o nvdiffrast nos commits usados, necessários para a imagem. Os demais modelos do benchmark e seus commits estão em `TERCEIROS.md`.

```
mkdir -p ~/cerise2026/gen3d/third_party && cd ~/cerise2026/gen3d/third_party
git clone https://github.com/Tencent-Hunyuan/Hunyuan3D-2.git Hunyuan3D-2
git -C Hunyuan3D-2 checkout f8db63096c8282cb27354314d896feba5ba6ff8a
git clone https://github.com/NVlabs/nvdiffrast.git nvdiffrast
git -C nvdiffrast checkout 253ac4fcea7de5f396371124af597e6cc957bfae
```

3. Baixe os pacotes do Release `dados-2026-10-26` (ou pela página Releases), confira o SHA-256 e extraia-os na raiz do repositório. O pacote `gen3d_dados_gso_cc0.tar.gz` traz o conjunto de avaliação, necessário ao benchmark e aos testes; `gen3d_servidor_brutos.tar.gz` traz as imagens da suíte de carga; os demais servem de referência (lista em `RELEASES.md`).

```
cd ~/cerise2026/gen3d
gh release download dados-2026-10-26 -R cerise-ufg/cerise-2026-gen3d -D ~/gen3d_releases
(cd ~/gen3d_releases && sha256sum -c SHA256SUMS)
for f in ~/gen3d_releases/gen3d_*.tar.gz; do tar -xzf "$f"; done
```

Sem o Release, `bash benchmark/gso_download.sh` baixa os objetos (CC-BY 4.0), `benchmark/prepare_gso.py`, no contêiner de desenvolvimento, renderiza `data/eval/`, e as fotos CC0 de `validation/real/` devem ser copiadas para `data/real/`.

3.4. Imagem do pipeline e pesos

1. Construa a imagem, que compila as extensões CUDA para `TORCH_CUDA_ARCH_LIST=12.1`, e confira-a.

```
cd ~/cerise2026/gen3d
sudo docker build -f docker/Dockerfile -t cerise26_gen3d:latest .
bash docker/smoke_image.sh
```

A conferência deve mostrar `cuda True`, `gen3d_pipeline ok; presets: ['rapido', 'padrao', 'alto']`, Node `v22.11.0` e glTF-Transform `4.5.1` (`resultados/docker_smoke.txt`).

2. Em x86_64 (não testado), antes do build, troque o Node para x64 e a arquitetura CUDA pela da sua GPU (8.9 no exemplo), também nos scripts `docker/install_*.sh`, que redefinem `TORCH_CUDA_ARCH_LIST` ao compilar as extensões de TripoSR, InstantMesh e Hunyuan3D-2.1 no benchmark, e confira se a imagem base tem variante amd64 no catálogo NGC.

```
sed -i -e 's/linux-arm64/linux-x64/g' \
  -e 's/ARCH_LIST="12.1"/ARCH_LIST="8.9"/' docker/Dockerfile
sed -i 's/ARCH_LIST="12.1"/ARCH_LIST="8.9"/' docker/install_*.sh
```

3. Baixe os pesos e monte a árvore `hy3dgen_models/`. O conjunto `hy2` traz forma, textura e *delight* do Hunyuan3D-2 e o 2mini; `clip` traz o CLIP das métricas. `make_local_tree.py` converte o UNet do módulo de textura para `.bin`, único formato aceito pelo carregador oficial, e `fix_turbo_unet.py` baixa o `.bin` do módulo turbo, cujo `.safetensors` tem chaves incompatíveis.

```
sudo docker run --rm --gpus all --ipc=host \
  -v ~/cerise2026/hf_cache:/root/.cache/huggingface \
  -v ~/cerise2026/gen3d/hy3dgen_models:/opt/gen3d/hy3dgen_models \
  cerise26_gen3d:latest bash -c "cd models \
  && python download_weights.py hy2 clip --hy3dgen-dir /opt/gen3d/hy3dgen_models \
  && python make_local_tree.py && python fix_turbo_unet.py"
```

Para o benchmark completo, acrescente `hy21 triposr instantmesh`. A árvore usa links absolutos: monte o cache sempre em `/root/.cache/huggingface`.

3.5. Geração de um objeto

1. Gere um GLB a partir de uma foto. O exemplo carrega os *presets* rápido e padrão: o `Gen3DPipeline` carrega forma e textura (~11,4 GB, cerca de 115 s) e aquece os dois *presets* (cerca de 80 s) uma vez, valores medidos com esses dois *presets* (`resultados/READY.json`); depois, cada geração leva dezenas de segundos.

```
mkdir -p /tmp/saida
sudo docker run --rm --gpus all --ipc=host \
  -v ~/cerise2026/hf_cache:/root/.cache/huggingface \
  -v ~/cerise2026/gen3d/hy3dgen_models:/opt/gen3d/hy3dgen_models \
  -v ~/cerise2026/gen3d/validation/real:/fotos:ro -v /tmp/saida:/saida \
  cerise26_gen3d:latest python -c "import sys; sys.path.insert(0,'pipeline'); \
from gen3d_pipeline import Gen3DPipeline, DEFAULT_PRESETS; \
p=Gen3DPipeline(presets={k: DEFAULT_PRESETS[k] for k in ['rapido','padrao']}); \
print(p.generate('/fotos/cc0_rooster.jpg','/saida','padrao')['stats'])"
```

2. Confira em `/tmp/saida` o `model.glb` (entregue ao app), os intermediários, `thumb.png` e `stats.json` (tempo por etapa, faces, tamanho, dimensões e pico de GPU). Sem o argumento `presets`, o `Gen3DPipeline()` carrega também o alto, que usa o módulo de textura completo; medido sozinho, o alto levou 107 s de carga e 118 s de aquecimento (`resultados/READY.json`).

Tabela 6. Presets de pipeline/presets.json (forma com 5 passos e FlashVDM) e tempo mediano no GSO (resultados/READY.json)

Preset	Octree	Faces	Textura gerada / entregue	Módulo de textura	Tempo mediano
rapido	256	10 mil	1024 / 1024 px	paint-v2-0-turbo	23,9 s
padrao	384	20 mil	2048 / 1024 px	paint-v2-0-turbo	30,4 s
alto	384	40 mil	2048 / 2048 px	paint-v2-0 completo	82,1 s

3.6. Benchmark A1 e medição dos presets

1. Crie o contêiner de desenvolvimento, com o repositório em `/workspace`, caminho esperado pelos scripts de `benchmark/` e `validation/`.

```
sudo docker run -d --name cerise26_gen3d_dev --gpus all --ipc=host \
  -e HY3DGEN_MODELS=/workspace/hy3dgen_models -e TP=/workspace/third_party \
  -v ~/cerise2026/gen3d:/workspace -v ~/cerise2026/hf_cache:/root/.cache/huggingface \
  -w /workspace/benchmark cerise26_gen3d:latest sleep infinity
```

2. Valide as métricas com o autoteste (malha de referência transformada) e rode o modelo escolhido nos 13 objetos, gravando numa pasta nova para preservar os resultados de referência.

```
D="sudo docker exec cerise26_gen3d_dev"
$D python selftest_metrics.py && $D python evaluate.py --tags gt_selftest
$D python run_model.py --config hy2_turbo --out /workspace/results/repro
$D python evaluate.py --results /workspace/results/repro --tags hy2_turbo
$D python aggregate.py --bench /workspace/results/repro --tags hy2_turbo \
  --prefix repro --out /workspace/results/repro --resdir /workspace/results/repro
```

O `evaluate.py` alinha cada malha à referência (ICP com escala), calcula Chamfer e F-score e mede o CLIP nas vistas renderizadas; o `aggregate.py` gera `repro_tabela.csv`, `repro_tabela.md` e os gráficos. As configurações ficam em `CONFIGS`, no `benchmark/run_model.py` (`hy2_turbo`, `hy2mini_turbo`, `hy2_full`, `hy21_full`, `triposr`, `instantmesh` e outras); `--objects` aceita uma lista separada por vírgulas e `--override` altera parâmetros em JSON.

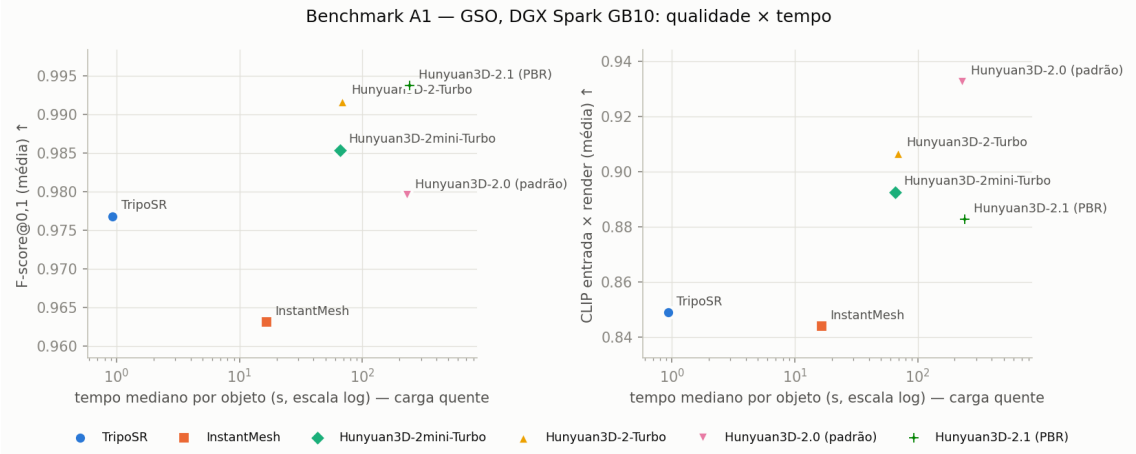


Figura 1. Resultado de referência do benchmark A1: F-score e CLIP em função do tempo mediano por objeto, com o Hunyuan3D-2 Turbo como melhor compromisso

3. TripoSR, InstantMesh e Hunyuan3D-2.1 precisam de dependências fora da imagem: clone-os em `third_party/` (commits em `TERCEIROS.md`) e rode, com `$D bash /workspace/docker/install_stage2b.sh`, `/workspace/docker/install_stage3.sh` e `/workspace/docker/install_hy21.sh`.

4. Meça os *presets* no pipeline empacotado (13 objetos e 6 fotos CC0); a saída regrava `results/presets/`.

```
$D python run_presets.py --presets rapido,padrao
$D python run_presets.py --presets alto
$D python evaluate.py --results /workspace/results/presets \
  --tags preset_rapido,preset_padrao,preset_alto
```

3.7. Testes funcionais A4

1. Rode a suíte pytest em duas partes, com cerca de 30 GB de memória livre. Ela verifica o glTF-Validator, a textura, a ausência de Draco, meshopt e

KTX2, as faces, a caixa de 0,30 m e os limites de tempo e tamanho; a saída fica em `results/a4_pytest/`.

```
sudo docker exec -e GEN3D_TEST_PRESETS=rapido,padrao -e JUNIT_NAME=rapido_padrao \
  cerise26_gen3d_dev bash /workspace/validation/run_a4.sh pytest
sudo docker exec -e GEN3D_TEST_PRESETS=alto -e JUNIT_NAME=alto \
  cerise26_gen3d_dev bash /workspace/validation/run_a4.sh pytest
```

2. Para o carregamento no `<model-viewer>` e no three.js (Chromium via Playwright), instale as ferramentas que faltam e rode as etapas que usam `results/presets/`; elas regravam figuras de `evidencias/` (compare com `git diff --stat`).

```
D="sudo docker exec cerise26_gen3d_dev"
$D bash /workspace/docker/install_gltftools.sh
$D bash -c "pip install --no-cache-dir playwright \
  && python -m playwright install --with-deps chromium"
$D bash /workspace/validation/run_a4.sh viewer
$D bash /workspace/validation/run_a4.sh grids
```

3.8. Servidor de geração 3D com fila

1. Construa a imagem do servidor e baixe o modelo de remoção de fundo do `rembg` 2.0.65, exigido por `scripts/start.sh`. O worker roda sem rede e com `HF_HUB_OFFLINE=1`; todos os pesos precisam estar no disco.

```
cd ~/cerise2026/gen3d/server
sudo docker build -t cerise26_gen3d_server:latest .
mkdir -p models/u2net
curl -L -o models/u2net/u2net.onnx \
  https://github.com/danielgatis/rembg/releases/download/v0.0.0/u2net.onnx
sha256sum models/u2net/u2net.onnx
```

O SHA-256 esperado é `8d10d2f3bb75ae3b6d527c77944fc5e7dcd94b29809d47a739a7a728a912b491`.

2. Ajuste os caminhos absolutos de `compose.yaml`, que apontam para a máquina original, e, se preciso, `GEN3D_CORS_ORIGINS` em `gen3d.env`.

```
sed -i "s#/home/[^/]*/cerise2026#$HOME/cerise2026#g" compose.yaml
grep -n cerise2026 compose.yaml
```

3. Suba, acompanhe e pare o serviço.

```
bash scripts/start.sh      # sobe API + worker e espera o /health "ok" (2 a 4 min)
bash scripts/status.sh     # contêineres, /health, fila, memória e GPU por processo
sudo docker logs -f cerise26_gen3d_worker1
bash scripts/stop.sh       # parada graciosa; a fila fica no SQLite em data/
```

A API escuta só em `127.0.0.1:8095`; o worker não tem rede. `bash scripts/start.sh 2` usa 2 *slots* com pesos compartilhados; `sudo touch data/PAUSE` drena a GPU sem derrubar a API; `bash scripts/redeploy.sh` reconstrói e recria tudo após mudanças no código. Os limites (fila de 32, até 4 pedidos em andamento por cliente, 6 envios por minuto por cliente e retenção de 7 dias) ficam em `gen3d.env` (`GEN3D_MAX_QUEUE`, `GEN3D_MAX_QUEUED_PER_CLIENT`, `GEN3D_RATE_JOBS_PER_MIN` e `GEN3D_RETENTION_DAYS`), descritos em `server/README.md`.

4. Rode os 18 testes automatizados, com motor simulado e sem GPU.

```
bash scripts/run_pytest.sh
```

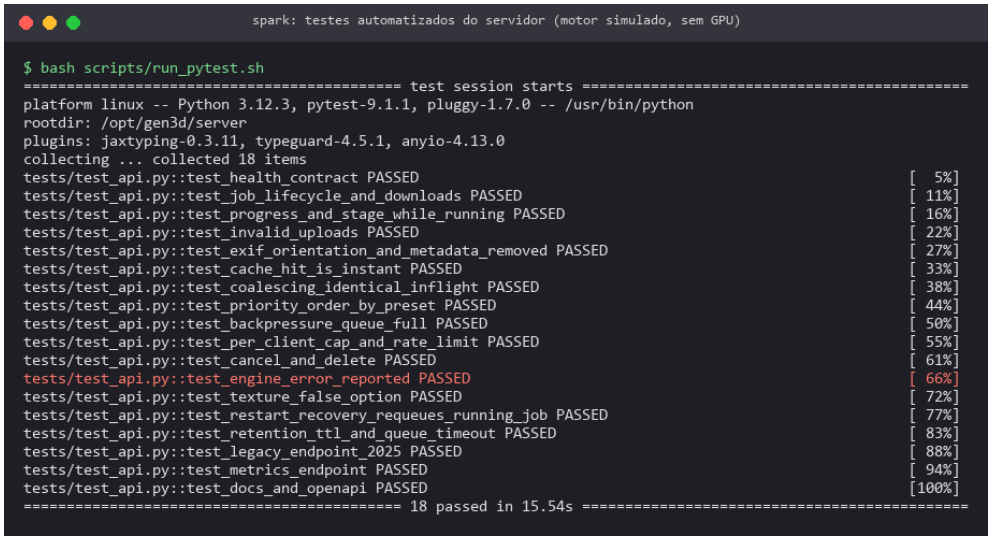


Figura 2. Saída esperada de scripts/run_pytest.sh: 18 testes do servidor aprovados

3.9. Testes com curl, endpoint legado e aplicativo

1. Na máquina do servidor, teste o fluxo com curl, trocando `<id>` pelo `job_id` devolvido.

```
curl -s http://127.0.0.1:8095/api/v1/health
curl -s -F "image=@../validation/real/cc0_rooster.jpg" \
  -F 'options={"preset":"rapido","texture":true,"remove_background":true}' \
  http://127.0.0.1:8095/api/v1/jobs
curl -s http://127.0.0.1:8095/api/v1/jobs/<id>
curl -s -o model.glb http://127.0.0.1:8095/api/v1/jobs/<id>/model.glb
curl -s -F "front=@../validation/real/cc0_rooster.jpg" -F 'metadata={}' \
  -o objeto_ajustado.glb http://127.0.0.1:8095/
```

O envio responde `202` com `job_id`, `position` e `eta_seconds`; o estado vai de `queued` a `running` (com `progress` e `stage`) e a `done`; reenviar a mesma foto responde na hora com `"cached":true`. A última chamada usa o endpoint legado de 2025 (`POST /`), que devolve o GLB na resposta. `bash scripts/smoke.sh ../validation/real/cc0_rooster.jpg padrao` faz o ciclo completo; a documentação interativa fica em `http://127.0.0.1:8095/docs`.

2. De outro computador, abra um encaminhamento local de SSH, que não expõe nada, e, em outro terminal, rode o teste ponta a ponta a partir da pasta `server` de um clone (requer Python com `requests`).

```
ssh -N -L 127.0.0.1:18095:localhost:8095 <usuario>@<maquina-gpu>
python scripts/e2e_windows.py --base http://127.0.0.1:18095 \
  --photo ../validation/real/cc0_rooster.jpg --out e2e_out
python scripts/glb_check.py e2e_out/model.glb
```

O `e2e_windows.py` percorre health, envio, fila, downloads, conferência do GLB, cache, endpoint legado e erros de validação; o `glb_check.py` mostra a caixa em metros, o formato da textura e as extensões.

3. No aplicativo CERISE AR 1.1.0 (repositório `cerise-ar`, branch `feat/2026-gen3d-playstore`), abra Configurações, informe a URL do servidor e toque em Testar conexão. No emulador Android, use `http://10.0.2.2:18095` com o túnel acima (`10.0.2.2` é o computador hospedeiro). Depois, escolha uma foto, acompanhe a fila e abra o modelo em Meus modelos. Sem GPU, `node cerise-expo/tools/mock-gen3d-server.js --port 18096`, do mesmo repositório, simula o contrato `/api/v1`.

3.10. Teste de carga

A suíte usa as 64 imagens de `server/loadtest/pool/`, do pacote `gen3d_servidor_brutos.tar.gz` ou geradas com `python3 loadtest/make_image_pool.py .. loadtest/pool` (requer Pillow). Com o servidor no ar:

```
cd ~/cerise2026/gen3d/server
bash loadtest/run_suite_new.sh
sudo docker stop cerise26_gen3d_worker1
bash loadtest/run_baseline.sh lock && bash loadtest/run_baseline.sh nolock
sudo docker start cerise26_gen3d_worker1
sudo docker run --rm -v "$PWD:/opt/gen3d/server" cerise26_gen3d_server:latest \
  python loadtest/analyze.py results/loadtest results/tabelas results/figuras
```

A suíte roda rajadas de 1 a 16 clientes com 1 e 2 *slots*, cache, coalescência, prioridade contra FIFO e *backpressure*, e no fim restaura a configuração de produção. O *baseline* reproduz o Flask síncrono de 2025 com o modelo real e exige o worker novo parado, por causa do teto de 24 GB.

3.11. Exposição pública (opcional)

O servidor não foi exposto à internet no projeto. O `server/README.md` traz uma proposta, que só deve ser aplicada com autorização do responsável pela máquina de GPU e com estes cuidados:

- prefira uma VPN entre o *proxy* público e a máquina de GPU; túneis SSH reversos são frágeis, e túneis de terceiros fazem as fotos passarem por outra empresa (LGPD);
- aceite só HTTPS, com TLS no *proxy*;
- gere suas próprias chaves em `GEN3D_API_KEYS`, num arquivo `gen3d.secrets.env` fora do git, incluído no `env_file` da API; o app envia `X-API-Key` quando compilado com `EXPO_PUBLIC_GEN3D_API_KEY`, mas uma chave embutida no APK só filtra abuso;
- use `GEN3D_TRUST_PROXY=1` só atrás de um *proxy* que sobrescreva o `X-Forwarded-For`;
- publique só `/api/v1/*` (e `POST /`, se necessário), bloqueie `/metrics`, `/docs` e `/openapi.json`, limite o corpo a 16 MB e mantenha a fila máxima e o *rate limit*.

3.12. Verificação do resultado

A máquina foi compartilhada com serviços de produção, e os tempos absolutos são limites superiores: o Hunyuan3D-2 Turbo levou 68,8 s por objeto sob carga plena e 28,7 s com a máquina parcialmente livre. As métricas de qualidade devem coincidir de perto.

Tabela 7. Valores de referência medidos e arquivos em que estão registrados

Verificação	Valor de referência	Arquivo
Autoteste das métricas	Chamfer médio 0,0042; F@0,05 ≥ 0,976	`results/bench/gt_selftest/metrics.csv`
A1, Hunyuan3D-2 Turbo	Chamfer 0,032; F@0,05 0,951; F@0,1 0,992; CLIP entrada 0,907; CLIP vistas novas 0,854; pico 14,6 GB	`resultados/a1_tabela.csv`
Presets (medianas no GSO)	23,9 s, 30,4 s e 82,1 s; pico 14,5 GB; carga 115 s e aquecimento 79 s	`resultados/READY.json`, `resultados/presets_tabela.csv`
A4	38 e 20 testes aprovados; 7 de 7 GLBs carregados	`resultados/a4_pytest/`, `resultados/viewer/viewer_report.json`
Testes do servidor	18 aprovados	`server/resultados/pytest_server_spark.txt`
Ponta a ponta (padrão)	GLB em 16,1 s; 20 mil triângulos; 0,72 MB; textura `image/jpeg`; maior dimensão 0,30 m	`server/resultados/e2e/e2e_console.txt`
Carga, fila e 1 worker (33 jobs)	33 de 33; p50 49,6 s e p95 174,1 s; 4,99 jobs/min com 16 clientes	`server/resultados/tabelas/carga_resumo.md`
Cache e *backpressure*	acerto em 0,02 s; fila máxima 8 e 16 clientes: 9 aceitos, 7 recusados com 429	`server/resultados/tabelas/carga_cenarios_extras.md`

Os tamanhos de GLB da rodada de *presets* (1,65, 2,01 e 6,14 MB) correspondem a texturas PNG, anteriores à correção do pós-processamento; com o código atual, saem bem menores (no padrão, a mesma foto passou de 2,97 MB para 0,72 MB).



Figura 3. Resultado esperado do teste ponta a ponta: foto enviada, miniatura do servidor, GLB com textura JPEG no model-viewer e, à direita, o mesmo modelo antes da correção da textura

3.13. Problemas comuns e soluções

- **Worker em loop de reinício após desligamento abrupto.** O cache do numba gravado no contêiner corrompeu. Mantenha `NUMBA_CACHE_DIR=/tmp/numba_cache` em `gen3d.env` e o `tmpfs` de `compose.yaml`, veja `sudo docker logs --tail 50 cerise26_gen3d_worker1` e recrie os contêineres, na pasta `server`, pois reiniciá-los não basta (ou rode `bash scripts/redeploy.sh`):

```
sudo GEN3D_WORKER_SLOTS=1 GEN3D_TRUST_PROXY=0 \
  docker compose -f compose.yaml up -d --force-recreate
```

- **GLB com textura PNG, maior que o esperado.** O `gltf-transform jpeg` 4.5 só recomprime texturas JPEG; o `pipeline/postprocess.py` atual passa `--formats "*"`. Reconstrua imagens antigas, confira `result.stats.texture_mime` e, sempre que a saída mudar, troque `GEN3D_PIPELINE_VERSION`, senão o cache devolve GLBs antigos.
- **Erro ao carregar o UNet do módulo de textura.** Rode de novo `make_local_tree.py` e `fix_turbo_unet.py`, com o cache em `/root/.cache/huggingface`.
- **Falta do modelo de remoção de fundo.** O `start.sh` acusa falta de `u2net.onnx` ou o worker tenta baixar pesos: o worker não tem rede, baixe tudo antes.
- **Worker não carrega o modelo ou não pega jobs.** Ele exige 10 GB de `MemAvailable` além do modelo (`GEN3D_MIN_AVAILABLE_GB`) e para enquanto existir `data/PAUSE`; sem o *preset* alto, economizam-se ~4,4 GB. Como o `compose.yaml` define `GEN3D_WORKER_PRESETS` no ambiente do worker, passe o valor na linha de comando, na pasta `server`:

```
sudo GEN3D_WORKER_SLOTS=1 GEN3D_TRUST_PROXY=0 GEN3D_WORKER_PRESETS=rapido,padrao \
  docker compose -f compose.yaml up -d --force-recreate worker1
```

- **HTTP 429 em testes manuais.** Aguarde o `Retry-After` ou, só para testes, aumente `GEN3D_RATE_JOBS_PER_MIN`.
- **Falhas com gerações simultâneas.** O *scheduler* da difusão guarda estado; para concorrência, use os *slots* do worker, que têm travas por componente.
- **Modelos bloqueados no benchmark.** TRELLIS e TRELLIS.2 dependem de `spconv` e `cumm`, sem pacotes para aarch64; Stable Fast 3D e SPAR3D exigem aceite da licença no Hugging Face. Em x86_64, não foram testados.
- **Extração no Windows.** Use `tar --force-local -xzf` quando o caminho começar com a letra da unidade.

3.14. Referências do repositório

- `README.md`, `RELEASES.md` e `TERCEIROS.md`: visão geral, pacotes com SHA-256 e componentes de terceiros.
- `RELATORIO_SECAO.md` e `RELATORIO_SECAO_SERVIDOR.md`: metodologia e resultados completos.
- `docker/Dockerfile`, `models/download_weights.py`, `models/make_local_tree.py` e `models/fix_turbo_unet.py`: imagem e pesos.
- `pipeline/gen3d_pipeline.py`, `pipeline/presets.json` e `pipeline/postprocess.py`: pipeline.
- `benchmark/run_model.py`, `benchmark/evaluate.py`, `benchmark/aggregate.py` e `benchmark/run_presets.py`: benchmark.
- `validation/test_pipeline.py` e `validation/run_a4.sh`: validação A4.
- `server/README.md`, `server/compose.yaml`, `server/gen3d.env`, `server/scripts/`, `server/tests/test_api.py` e `server/loadtest/`: servidor.

4. APLICATIVO CERISE AR COM GERAÇÃO DE MODELOS 3D

4.1. O que será reproduzido

Este roteiro mostra como compilar, executar e preparar para publicação o aplicativo CERISE AR 1.1.0 (Expo/React Native), que fotografa um objeto, envia a foto ao servidor de geração 3D do CERISE, acompanha a fila, baixa o modelo GLB texturizado e o exibe num visualizador 3D e em realidade aumentada (RA). Ele cobre a Atividade 2 do Objetivo 1 (versão funcional do aplicativo com geração de objetos 3D texturizados, lado do aplicativo) e a Atividade 6 do Objetivo 2 (adequação à Play Store). O servidor de geração 3D tem roteiro próprio (repositório `cerise-2026-gen3d`); aqui ele é usado pronto ou substituído por um servidor simulado (*mock*).

Ao final, você terá:

- o AAB e o APK release do Android assinados com a sua própria chave de *upload*, mais um APK para o emulador x86_64;
- o app rodando no emulador Android (e, se tiver, num aparelho ARM) e no Simulador iOS;
- o fluxo foto → modelo 3D → Meus modelos → visualizador 3D testado contra o *mock* e, se houver acesso, contra o servidor real;

- opcionalmente, um servidor de atualizações OTA próprio e o pacote da Play Store pronto para envio.

4.2. Requisitos

- Computador Windows x86_64 (o projeto usou um notebook com Windows 11) com virtualização habilitada. Compilar o Android no Linux ou no macOS com `./gradlew` não foi testado.
- Mac com Apple Silicon para o iOS (o projeto usou um Mac mini M4 com macOS 26.3.1 e Xcode 26.6).
- Opcional: aparelho Android ARM com ARCore e iPhone com ARKit (iOS 17.6 ou superior), pois a RA não roda em emulador nem em simulador.
- Servidor 3D real: no projeto, rodou numa NVIDIA DGX Spark (GPU GB10, CPU aarch64, 128 GB de memória unificada, Ubuntu 24.04, Docker). O app não depende dessa arquitetura; a adaptação do servidor para x86_64 com GPU NVIDIA comum pertence ao roteiro do servidor e não foi testada aqui. Sem ele, use o *mock*, que roda em qualquer máquina com Node.js.

Tabela 8. Software usado na reprodução do aplicativo

Software	Versão	Uso
Git e GitHub CLI (`gh`)	atual	clonar o repositório privado e baixar as *Releases*
Node.js e npm	LTS (versão exata não registrada)	dependências do app, *mock*, scripts
JDK	17	Gradle do Android
Android SDK	API 36, *build-tools* (zipalign, apksigner, aapt2), emulador	compilação e verificação
Imagem do emulador	Android 16, API 36, x86_64 com Google Play (AVD `Medium_Phone_API_36.0`)	testes no Windows
Xcode, CocoaPods, Homebrew	Xcode 26.6	compilação iOS
Python 3 com Pillow	3.11	verificador de 16 KB e gráficos da loja
Docker	atual	servidor OTA (opcional)

As dependências do app estão fixadas em `cerise-expo/package-lock.json` (Expo SDK 57, React Native 0.86.3, React 19.2.3, ViroReact 2.58, expo-updates).

Contas: acesso à organização `cerise-ufg` no GitHub (repositório privado, solicitado ao bolsista); para publicar, conta de desenvolvedor do Google Play (taxa única de US$ 25), Apple Developer Program (TestFlight) e um domínio com HTTPS.

Espaço e tempo: o build iOS consumiu cerca de 4 GB no Mac (`docs/mac-ios/logs/build-ios-sim.log`), com `npm ci` em 5 s, `pod install` em 46 s e `xcodebuild` em 72 s (`docs/relatorio-2026/resultados/ios-build-resumo.txt`). O primeiro build Gradle não foi cronometrado; como estimativa, reserve 20 GB no Windows e um dia de trabalho para Android e iOS.

4.3. Obtenção do código e dos dados

1. Peça acesso ao repositório e autentique o GitHub CLI com `gh auth login`.

2. Clone o repositório e mude para o branch da versão 1.1.0. O `main` contém o app nativo 1.0.0 (Kotlin e SwiftUI) e o app Expo 1.0.0; as pastas `cerise-expo/plugins/`, `store/`, `tools/` e `src/gen3d/` existem só no branch `feat/2026-gen3d-playstore`.

```
git clone https://github.com/cerise-ufg/cerise-ar.git
cd cerise-ar
git switch feat/2026-gen3d-playstore
```

3. Baixe, se quiser, os artefatos de referência: a *release* `v1.1.0` traz o AAB, o APK universal e o APK de emulador; a `v1.0.0` traz os IPAs, os `.xcarchive` e o bundle OTA de setembro de 2026.

```
gh release download v1.1.0 -R cerise-ufg/cerise-ar
gh release download v1.0.0 -R cerise-ufg/cerise-ar \
  -p cerise-ar-ota-update-1.0.0-2026-09-18.zip
sha256sum cerise-ar-1.1.0-110.aab
```

4. Compare cada SHA-256 com a tabela de `RELEASES.md` (no Windows: `Get-FileHash .\cerise-ar-1.1.0-110.aab -Algorithm SHA256`). Os `.tar` do iOS se extraem com `tar -xf`; o bundle OTA, com `unzip`, que cria `dist/`.

5. Fotos de teste: use as fotos CC0 de `validation/real/` do repositório `cerise-2026-gen3d` (`cc0_duck.jpg`, `cc0_teapot.jpg`, `cc0_cat.jpg`, `cc0_donkey.jpg`) ou `docs/mac-ios/amostras/` deste repositório.

4.4. Dependências e servidor de teste

1. Instale as dependências exatamente como no `package-lock.json`:

```
cd cerise-expo
npm ci
```

2. Suba o *mock* do servidor 3D. Ele reproduz o contrato `/api/v1` (fila com um *worker*, posição, ETA, estágios, progresso, cancelamento) e devolve GLBs de exemplo do catálogo, sem gerar nada a partir da foto; o app marca esses resultados como "servidor MOCK". Por padrão, escuta só em 127.0.0.1.

```
node tools/mock-gen3d-server.js --port 18096
# opcionais: MOCK_SPEED=0.5 (2x mais rápido), MOCK_FAIL_EVERY=3, MOCK_MAX_QUEUE=8
```

O log deve começar com `mock gen3d ouvindo em http://127.0.0.1:18096 (4 GLBs de exemplo, speed x1)`, e `GET /api/v1/health` deve devolver `"status":"ok"` e `"model":"mock (GLB de exemplo, sem IA)"`.

3. Servidor real (opcional): ele escuta só no *loopback* da máquina de GPU, na porta 8095. Para alcançá-lo do computador do emulador sem expor nada, use um encaminhamento SSH local:

```
ssh -N -L 127.0.0.1:18095:localhost:8095 <usuario>@<maquina-gpu>
curl http://127.0.0.1:18095/api/v1/health
```

4.5. Compilação e execução no Android

1. Gere a sua chave de *upload* (as do projeto não são distribuídas). Dentro de `cerise-expo`, crie a pasta `keystore/` (ignorada pelo git), gere o keystore e escreva `keystore/upload.properties` no formato lido pelo *plugin* `plugins/with-upload-signing.js`. Guarde o keystore e as senhas num cofre.

```
mkdir keystore
keytool -genkeypair -v -keystore keystore\cerise-upload.jks -alias cerise-upload `
  -keyalg RSA -keysize 4096 -validity 10000

storeFile=cerise-upload.jks
storePassword=<senha do keystore>
keyAlias=cerise-upload
keyPassword=<senha da chave>
```

2. Gere o projeto nativo a partir do `app.json` e dos *plugins*, recrie o `local.properties` (o `--clean` apaga a pasta `android/`) e crie uma junção de caminho curto, porque o caminho do projeto estoura o limite de 260 caracteres do Windows no *codegen* C++:

```
npx expo prebuild -p android --clean --no-install
Set-Content android\local.properties `
  'sdk.dir=C\:\\Users\\<usuario>\\AppData\\Local\\Android\\Sdk'
cmd /c mklink /J C:\cx <caminho-do-clone>\cerise-ar\cerise-expo    # uma vez
```

3. Compile o AAB e o APK release pela junção e copie-os para `build-out/` (ignorada pelo git):

```
cd C:\cx\android
.\gradlew.bat :app:bundleRelease :app:assembleRelease
mkdir ..\build-out
copy app\build\outputs\bundle\release\app-release.aab ..\build-out\cerise-ar-1.1.0-110.aab
copy app\build\outputs\apk\release\app-release.apk ..\build-out\cerise-ar-1.1.0-110.apk
```

4. Gere o APK de emulador. O app só tem bibliotecas ARM (`armeabi-v7a` e `arm64-v8a`, as únicas do ViroReact, fixadas no `app.json`); o emulador x86_64 as executa por tradução, mas só as encontra se forem extraídas na instalação. O comando sobrescreve o APK anterior, por isso copie-o antes:

```
.\gradlew.bat :app:assembleRelease '-Pexpo.useLegacyPackaging=true'
copy app\build\outputs\apk\release\app-release.apk `
  ..\build-out\cerise-ar-1.1.0-110-emulador-legacypackaging.apk
```

5. No Device Manager do Android Studio, crie um AVD Medium Phone com Android 16 (API 36), Google Play e x86_64, inicie-o e instale o APK adequado (num aparelho ARM, o universal):

```
adb install C:\cx\build-out\cerise-ar-1.1.0-110-emulador-legacypackaging.apk  # emulador
adb install C:\cx\build-out\cerise-ar-1.1.0-110.apk                            # aparelho ARM
adb logcat -s ReactNativeJS
```

O `logcat` mostra, ao fim de cada download, a linha `[gen3d] modelo pronto {...}` com os tempos de envio, fila, geração e download medidos pelo app.

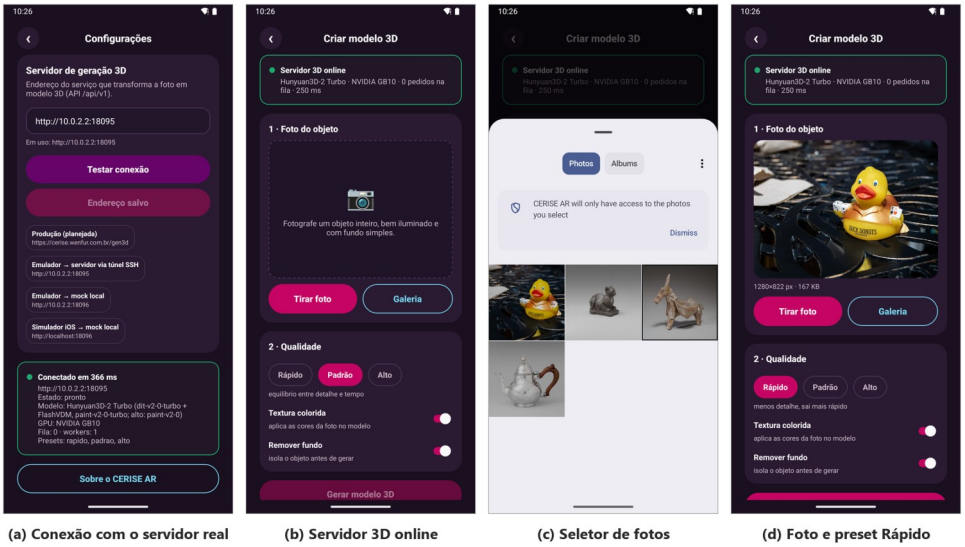
4.6. Configuração do servidor e teste do fluxo

1. No app, abra Configurações e informe a URL: `http://10.0.2.2:18096` (emulador → *mock*), `http://10.0.2.2:18095` (emulador → servidor real pelo encaminhamento) ou `http://localhost:18096` (Simulador iOS → *mock*). Toque em Testar conexão: deve aparecer "Conectado", com estado, modelo, GPU, fila e *presets*.

2. Em Criar modelo 3D, escolha a foto pela câmera ou pela galeria (no emulador, arraste a foto para a janela para copiá-la), a qualidade (Rápido, Padrão ou Alto) e as opções Textura colorida e Remover fundo, e toque em Gerar modelo 3D.

3. Acompanhe a posição na fila, a ETA e o progresso por estágio. O card Pronto mostra tamanho, vértices, triângulos e tempo total, com Ver em 3D e Ver em RA.

4. Em Meus modelos, abra os detalhes (tempos no app e, com o servidor real, "No servidor: fila X s · GPU Y s") e o visualizador 3D, que funciona sem rede. A RA exige aparelho físico. Excluir um modelo também apaga a foto e o modelo no servidor.



(a) Conexão com o servidor real (b) Servidor 3D online (c) Seletor de fotos (d) Foto e preset Rápido

Figura 4. App no emulador contra o servidor real: teste de conexão, servidor 3D online, seletor de fotos e escolha do preset

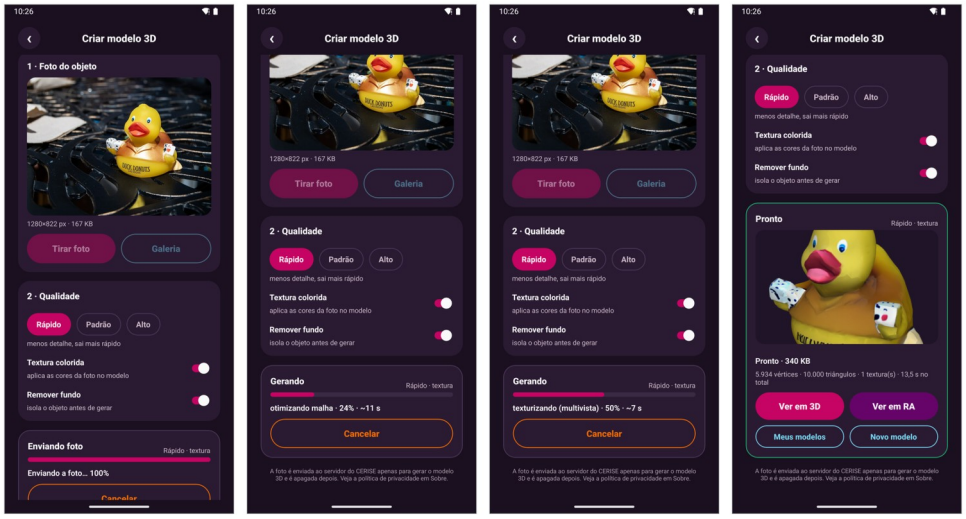


Figura 5. Fluxo de geração com o servidor real: envio da foto, geração da malha, texturização e resultado em 13,5 s no total

O build release bloqueia `http://` para qualquer host exceto `10.0.2.2`, `localhost` e `127.0.0.1` (*plugin* `plugins/with-network-security.js`); num aparelho físico, use um servidor com HTTPS. Para um teste de laboratório, nunca no build da loja, o *plugin* aceita a opção `hosts` no `app.json` e o *mock* aceita `--host 0.0.0.0`; esse caso não foi testado no projeto.

4.7. Compilação e execução no iOS

1. Simulador. O `ViroKit.framework` só tem a fatia arm64 de dispositivo, por isso o build de simulador usa `CERISE_NO_VIRO=1`, que remove o módulo de RA (o app esconde as entradas de RA). O script do projeto supõe o clone em `~/CeriseAR`, compila para o iPhone 17 Pro (troque o `-destination` se o seu Xcode não tiver esse simulador) e grava o `.app` em `~/CeriseAR/build-sim-dd`:

```
gh auth login                  # no Mac, se ainda não autenticou
gh repo clone cerise-ufg/cerise-ar ~/CeriseAR
cd ~/CeriseAR && git switch feat/2026-gen3d-playstore
bash cerise-expo/tools/ios/build-ios-sim.sh
```

O equivalente manual, dentro de `~/CeriseAR/cerise-expo`, é o abaixo. Mantenha `CERISE_NO_VIRO=1` exportada em todos os passos: o `react-native.config.js` a lê no *autolinking*, que roda no `pod install`, e sem ela o ViroReact volta a ser ligado. O `-derivedDataPath` grava o `.app` no caminho usado no passo 2.

```
export CERISE_NO_VIRO=1
npm ci
npx expo prebuild -p ios --clean --no-install
(cd ios && pod install)
xcodebuild -workspace ios/CERISEAR.xcworkspace -scheme CERISEAR -sdk iphonesimulator \
  -configuration Release -destination 'platform=iOS Simulator,name=iPhone 17 Pro' \
  -derivedDataPath ~/CeriseAR/build-sim-dd CODE_SIGNING_ALLOWED=NO build
```

2. Abra o Simulador (Xcode → Open Developer Tool → Simulator), instale e abra o app e suba o *mock* no Mac; depois configure `http://localhost:18096` e repita o teste do fluxo (arraste uma foto para o Simulador para adicioná-la à galeria):

```
xcrun simctl install booted \
  ~/CeriseAR/build-sim-dd/Build/Products/Release-iphonesimulator/CERISEAR.app
xcrun simctl launch booted br.ufg.cerise.arviewer
node ~/CeriseAR/cerise-expo/tools/mock-gen3d-server.js --port 18096
```

Para refazer as capturas do projeto sem interação, copie `docs/mac-ios/amostras` para `~/CeriseAR/samples` e rode `bash cerise-expo/tools/ios/rebuild-ios-sim.sh` (recompila com `CERISE_TEST_MODE=1`, que habilita os links `cerisear://`) e `bash cerise-expo/tools/ios/capture-ios.sh` (telas em `~/CeriseAR/shots-ios`).

3. iPhone e TestFlight. Troque `ios.bundleIdentifier` no `app.json` por um identificador seu (`br.ufg.cerise.arviewer` pertence à conta do projeto) e, na sessão gráfica do Mac, gere o projeto com RA:

```
cd ~/CeriseAR/cerise-expo
unset CERISE_NO_VIRO CERISE_TEST_MODE
npx expo prebuild -p ios --clean
(cd ios && pod install)
open ios/CERISEAR.xcworkspace
```

No Xcode, em CERISEAR → Signing & Capabilities, marque Automatically manage signing e escolha a sua equipe; selecione o iPhone, a configuração Release (Product → Scheme → Edit Scheme) e Product → Run. Para o TestFlight: destino Any iOS Device (arm64), Product → Archive e, no Organizer, Distribute App → App Store Connect → Upload; no App Store Connect, adicione testadores internos na aba TestFlight. Detalhes em `cerise-expo/store/IOS_IPHONE_TESTFLIGHT.md` e `ios/TESTFLIGHT.md`. Na versão 1.1.0, essa etapa foi documentada, mas não executada.

4.8. Servidor OTA próprio (opcional)

O app usa o protocolo expo-updates com servidor próprio, sem EAS. A `runtimeVersion` é a `version` do `app.json` (1.1.0), e só JavaScript e *assets* vão por OTA.

1. Construa e suba o servidor (Node/Express, porta 3000), com os dados num volume e a URL pública em `PUBLIC_BASE`, atrás de um *proxy* HTTPS (o `ota-server/README.md` traz a regra do Caddy: `handle_path /updates/*` → `reverse_proxy cerise-ota-server:3000`):

```
docker build -t cerise-ota-server ota-server/
docker run -d --name cerise-ota-server -p 3000:3000 -v <pasta-de-dados>:/data \
  -e PUBLIC_BASE=https://<seu-dominio>/updates cerise-ota-server
curl -i -H "expo-runtime-version: 1.1.0" -H "expo-platform: android" \
  http://localhost:3000/manifest
```

Sem publicações, o manifesto responde 204. Para testar sem compilar, copie o conteúdo do `dist/` da *release* `v1.0.0` para `<pasta-de-dados>/updates/1.0.0/<nome-qualquer>/` e repita o `curl` com `expo-runtime-version: 1.0.0`: a resposta passa a 200, com o `launchAsset`.

2. No `app.json`, troque `updates.url` por `https://<seu-dominio>/updates/manifest` e recompile (mudança nativa). No `ota-server/publish-update.mjs`, ajuste `SSH_HOST` (o seu servidor) e `REMOTE_ROOT` (`<pasta-de-dados>/updates`) e publique, na raiz do repositório:

```
node ota-server/publish-update.mjs "o que mudou"
```

O script executa `npx expo export --platform all --clear`, grava `dist/update.json` e copia `dist/` para `REMOTE_ROOT/<runtimeVersion>/<id>/`; os apps com a mesma `runtimeVersion` recebem a atualização na próxima abertura. O placar do jogo Caça CERISE usa o mesmo servidor (`SCORES_URL` em `src/screens/GameScreen.tsx`).

4.9. Preparação da publicação na Play Store

1. No `app.json`, ajuste `android.package` (o `com.cerisear` pode já estar em uso na loja), `version` e `android.versionCode` (que cresce a cada envio) e rode de novo `npx expo prebuild -p android --clean --no-install`.

2. Publique o servidor 3D com HTTPS e defina a URL padrão na constante `DEFAULT_SERVER` de `src/gen3d/settings.ts`. Se o servidor exigir chave (`GEN3D_API_KEYS`), defina `$env:EXPO_PUBLIC_GEN3D_API_KEY = '<sua chave>'` antes do Gradle; a chave fica embutida no bundle e só protege contra uso casual.

3. Adapte `store/privacidade.html` (controlador, contato, retenção), publique-a numa URL HTTPS e atualize `PRIVACY_URL` em `src/screens/AboutScreen.tsx` e `extra.privacyPolicyUrl` no `app.json`.

4. Revise a ficha `store/listing-pt-BR.md`, confira os limites de caracteres e, se trocar as capturas, regenere ícone, recurso gráfico e capturas (na raiz do repositório):

```
node cerise-expo/store/check-listing.js
python cerise-expo/store/make-graphics.py docs/relatorio-2026/evidencias
```

5. Recompile e verifique o AAB (seção seguinte) e siga as etapas 1 a 7 de `store/PUBLICAR_PLAY_STORE.md`, com as respostas de `store/formularios-play-console.md`: criar o app; configurar política, acesso, anúncios, IARC, público-alvo (13+) e segurança dos dados; preencher a ficha; enviar o AAB ao teste interno, aceitando o Play App Signing; fazer o teste fechado (12 testadores por 14 dias, exigido de contas pessoais criadas após novembro de 2023); e só então pedir a produção, com o servidor 3D respondendo na URL pública. Os passos 0b e 0c do guia são específicos do servidor do projeto.

4.10. Verificação do resultado

Em `C:\cx`, com a pasta *build-tools* do SDK no PATH, verifique os artefatos:

```
python tools/check-16kb.py build-out/cerise-ar-1.1.0-110.aab `
  build-out/cerise-ar-1.1.0-110.apk
zipalign -c -P 16 -v 4 build-out/cerise-ar-1.1.0-110.apk
aapt2 dump badging build-out/cerise-ar-1.1.0-110.apk
apksigner verify --print-certs build-out/cerise-ar-1.1.0-110.apk
jarsigner -verify build-out/cerise-ar-1.1.0-110.aab
```

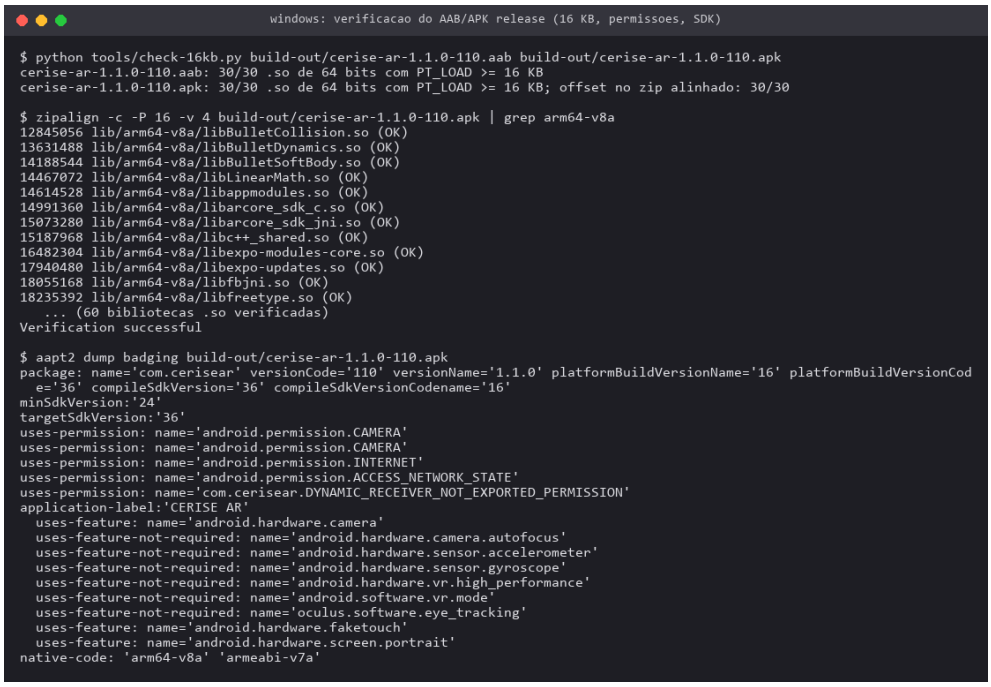


Figura 6. Verificação do AAB e do APK release: páginas de 16 KB, zipalign, SDK alvo 36, permissões e ABIs

Os tamanhos do seu build devem ser próximos dos de referência, e o certificado será o da sua chave. Na tabela, `resultados/` é `docs/relatorio-2026/resultados/`.

Tabela 9. Valores de referência medidos no projeto

Verificação	Resultado esperado	Arquivo
`check-16kb.py`	AAB e APK: 30/30 `.so` de 64 bits com PT_LOAD ≥ 16 KB; offset no zip alinhado: 30/30	`resultados/final-check-16kb.json`
`zipalign -c -P 16`	Verification successful	`resultados/final-artefatos-verificacao.txt`
`aapt2 dump badging`	versionCode 110, versionName 1.1.0, minSdk 24, targetSdk 36; CAMERA, INTERNET, ACCESS_NETWORK_STATE; ABIs arm64-v8a e armeabi-v7a	`resultados/apk-badging.txt`
Tamanhos	AAB 55.157.760 B; APK 83.299.939 B; APK de emulador 48.616.475 B	`resultados/final-artefatos-verificacao.txt`

Verificação	Resultado esperado	Arquivo
Simulador iOS	BUILD SUCCEEDED; `.app` de 41 MB, 1.1.0 build 2, iOS mínimo 17.6	`resultados/ios-build-resumo.txt`
Fluxo com o *mock*	geração simulada de 12 s, 18 s e 26 s (Rápido, Padrão e Alto, com textura); Padrão: 23,2 s no total	`tools/mock-gen3d-server.js`; Tabela 3 de `RELATORIO_SECAO.md`
Fluxo com o servidor real, sem fila	total de 13,5 s (Rápido), 17,1 s (Padrão) e 31,5 s (Alto); reenvio da mesma foto pelo cache: 1,0 s	`resultados/real-tempos-app.csv`
GLB recebido	textura JPEG, maior dimensão 0,300 m; 10.000, 19.998 e 39.998 triângulos por *preset*	`resultados/real-tempos-app.csv`
Exclusão no app	`GET /api/v1/jobs/{id}` passa de 200 a 404	`resultados/real-exclusao-servidor.txt`

Para regenerar a tabela de tempos a partir dos registros brutos, execute na raiz `python docs/relatorio-2026/resultados/scripts/tempos_e2e.py docs/relatorio-2026/resultados/real-logcat-app.txt docs/relatorio-2026/resultados/real-jobs-servidor.json`; o `real-tempos-app.csv` gerado é idêntico ao versionado.

4.11. Problemas comuns e soluções

- Erro de caminho longo no *codegen* C++ do Gradle no Windows: compile sempre pela junção `C:\cx` (limite de 260 caracteres).
- "SDK location not found" após `expo prebuild --clean`: a pasta `android/` foi recriada sem o `local.properties`; grave-o de novo.
- Aviso `[cerise] keystore/upload.properties ausente`: o release foi assinado com a chave de depuração; crie o `upload.properties`.
- App fecha ao abrir no emulador x86_64: instale o APK de emulador (`-Pexpo.useLegacyPackaging=true`), porque o `SoLoader` não encontra as bibliotecas ARM não comprimidas sob tradução. O build da loja mantém o empacotamento padrão.
- Não acrescente `x86` ou `x86_64` às ABIs: o ViroReact não tem essas bibliotecas, e o app encerrava com `UnsatisfiedLinkError: libviro_renderer.so`.
- RA no emulador: com o Google Play Services for AR instalado, o ARCore x86 não carrega no processo ARM traduzido (`AR_ERROR_FATAL`) e o ViroReact encerra o app; sem ele, o app mostra "RA indisponível". Teste a RA em aparelho físico.
- ViroKit não linka no Simulador iOS: use `CERISE_NO_VIRO=1`. Para iPhone, faça `unset CERISE_NO_VIRO` e um novo `npx expo prebuild -p ios --clean`, senão o app vai sem RA.

- Assinatura iOS por SSH falha com `errSecInternalComponent`: o *keychain* fica travado em sessões SSH; assine na sessão gráfica ou destrave-o (Opção C de `ios/INSTALL-IPHONE-MAC.md`).
- Páginas de 16 KB: a Play as exige para apps com alvo Android 15 ou superior; se o `check-16kb.py` listar uma biblioteca fora do padrão, atualize a dependência que a fornece.
- "Servidor 3D não respondeu" num aparelho físico: o release bloqueia `http://` fora dos endereços de teste; use HTTPS. Avisos "Servidor 3D iniciando" ou "em pausa" indicam que o pedido espera o *worker* carregar o modelo.
- A atualização OTA não chega: o app só aceita a mesma `runtimeVersion` (a `version` do `app.json`); mudanças nativas exigem novo build.

4.12. Referências do repositório

- `README.md` e `RELEASES.md`: visão geral, branches e artefatos com SHA-256.
- `cerise-expo/README.md`, `app.json` e `app.config.js`: contrato da API, builds, `CERISE_NO_VIRO` e `CERISE_TEST_MODE`.
- `cerise-expo/plugins/`: assinatura de *upload* e bloqueio de `http://`.
- `cerise-expo/src/gen3d/`: cliente da API, fila, download e URL padrão (`settings.ts`).
- `cerise-expo/tools/`: *mock*, verificador de 16 KB, gerador do visualizador e scripts do iOS.
- `cerise-expo/store/`: guias da Play Store e do TestFlight, formulários, ficha e política de privacidade.
- `ota-server/`: servidor OTA, Dockerfile e script de publicação.
- `docs/relatorio-2026/`: texto técnico (`RELATORIO_SECAO.md`), figuras (`evidencias/`) e dados medidos (`resultados/`).
- `ios/INSTALL-IPHONE-MAC.md` e `ios/TESTFLIGHT.md`: assinatura e envio no Mac.

5. DATASET SINTÉTICO, MODELO DIGITAL E GÊMEO DIGITAL DO BRAÇO ROBÓTICO

5.1. O que será reproduzido

Este roteiro reproduz o trabalho com o braço robótico virtual servo6dof: a geração de um dataset sintético multimodal no Ignition Gazebo Fortress, o modelo digital do braço (URDF, cinemática direta validada e planejamento com

MoveIt 2) e o gêmeo digital, que mostra no navegador ou no RViz um segundo braço acompanhando as juntas estimadas a partir da imagem. Tudo roda em contêineres Docker com ROS 2 Humble, a partir do repositório `cerise-2026-servo6dof-gemeo-digital` da organização `cerise-ufg`. Ao final, você terá:

- o dataset da Atividade 7 do Objetivo 3: 5.000 poses, 30.000 imagens 1280×720 (RGB, profundidade e segmentação de duas câmeras) e anotações JSON, baixado das Releases (versões cinza e laranja) ou regenerado, com `index.csv` e verificação de integridade;
- o modelo digital da Atividade 11 do Objetivo 4: URDF gerado do xacro, tabelas de links e juntas, cinemática direta comparada ao TF do ROS 2, espaço de trabalho, IoU entre modelo e imagens e MoveIt 2 sem interface gráfica;
- o gêmeo digital da Atividade 12 do Objetivo 4: servidor FastAPI/WebSocket com front-end three.js no modo verdade-base (GT) e no modo com o modelo de pose, além do nó ROS 2 que publica `/twin/joint_states` para o RViz.

5.2. Requisitos

Tabela 10. Requisitos para reproduzir o dataset, o modelo digital e o gêmeo digital

Item	Usado no projeto	Observações
Computador com GPU	NVIDIA DGX Spark: GPU GB10 (Blackwell, sm_121), CPU aarch64, 128 GB de memória unificada	Em x86_64 com GPU NVIDIA comum, aplique o ajuste da Etapa 1 (não testado). Com menos memória ou VRAM, use menos workers do Gazebo (o consumo por worker não foi medido)
Sistema	Ubuntu 24.04, Docker e NVIDIA Container Toolkit	Os scripts chamam `sudo docker`; o driver precisa oferecer EGL sem tela (headless)
Software nos contêineres	ROS 2 Humble (`ros:humble-ros-base`), Ignition Gazebo Fortress, `ros-humble-ign-ros2-control`, MoveIt 2 (OMPL e KDL), FastAPI 0.115.6, uvicorn 0.32.1, Playwright 1.49.1; three.js 0.160.0 e urdf-loader 0.13.1 já vêm no repositório	Instalados pelos Dockerfiles de `docker/`
Software no host	git, GitHub CLI (`gh`), Python 3 (só biblioteca padrão), `sha256sum`, `tar`, `curl`	`ffmpeg` só para as animações
Modo com o modelo de pose	repositório `cerise-2026-pose-servo6dof` e imagem `cerise26_pose:latest` (base `nvcr.io/nvidia/pytorch:26.05-py3`)	Pesos do Depth Anything V2 Large baixados do Hugging Face
Contas e acessos	conta no GitHub com acesso à organização `cerise-ufg`	Repositórios privados: solicite acesso ao bolsista e autentique com `gh auth login`
Disco	Releases: 5,16 GB no total (2,58 GB só a versão cinza); cada versão extraída ocupa 2,51 GB	A imagem `cerise26_pose` tem cerca de 35,5 GB; reserve ao menos 60 GB livres (estimativa)
Tempo	builds: cerca de 4 min (`servo6dof:humble`) e 1 min (`cerise26_gemeo`) no Spark; geração completa: 3,64 h com 8 workers	Logs dos builds em `resultados/logs_spark/`

5.3. Obtenção do código e dos dados

Os scripts assumem o repositório em `~/cerise2026/gemeo/repo`, a pasta do gerador em `~/braco` (dataset em `~/braco/dataset`) e os metadados em `~/cerise2026/gemeo/dataset_meta`. As variáveis `REPO`, `DATASET` e `META` de `scripts/rodar_container.sh` e de `gemeo_digital/subir_servidor.sh` trocam esses caminhos, mas `dataset_tools/analise/rodar_analise.sh` e `gemeo_digital/ros2/rodar_rviz_pose.sh` usam caminhos fixos; mantenha o layout.

1. Autentique a GitHub CLI (escolha HTTPS e aceite autenticar o Git com as suas credenciais do GitHub) e clone o repositório:

```
gh auth login
mkdir -p ~/cerise2026/gemeo ~/braco
gh repo clone cerise-ufg/cerise-2026-servo6dof-gemeo-digital ~/cerise2026/gemeo/repo
```

2. Baixe o dataset do Release `dados-2026-10-26` (ou pela página Releases do repositório) e confira os SHA-256. O comando baixa só a versão cinza, usada em todos os números de referência; sem as opções `-p` e `--ignore-missing`, baixa e confere tudo (variante laranja e quadros brutos do gêmeo).

```
cd ~/braco
gh release download dados-2026-10-26 -R cerise-ufg/cerise-2026-servo6dof-gemeo-digital \
  -D releases -p 'dataset_cinza.*' -p SHA256SUMS.txt
(cd releases && sha256sum -c --ignore-missing SHA256SUMS.txt)
```

3. Extraia as duas partes (independentes; juntas formam a pasta completa), confira cada arquivo contra o manifesto e coloque a pasta no lugar esperado:

```
cd ~/braco
tar -xf releases/dataset_cinza.part01.tar
tar -xf releases/dataset_cinza.part02.tar
sha256sum -c --quiet releases/dataset_cinza.manifest.sha256 && echo MANIFESTO_OK
ls dataset_cinza | wc -l          # 35009
mv dataset_cinza dataset
```

Para a variante laranja, use `-p 'dataset_laranja.*'`, extraia da mesma forma e aponte `DATASET=~/braco/dataset_laranja` ao chamar `rodar_container.sh` ou `subir_servidor.sh`. O formato dos arquivos está em `docs/DATASET.md`; uma amostra de 8 poses fica em `amostra_dataset/`.

5.4. Etapa 1: imagens Docker

1. Construa as imagens. A cadeia `servo6dof:humble` → `cerise26_gemeo` → `cerise26_gemeo_rviz` exige essa ordem:

```
cd ~/cerise2026/gemeo/repo
sudo docker build -f docker/Dockerfile.spark -t servo6dof:humble docker
sudo docker build -f docker/Dockerfile.gemeo -t cerise26_gemeo:latest docker
sudo docker build -f docker/Dockerfile.playwright -t cerise26_gemeo_playwright:latest docker
sudo docker build -f docker/Dockerfile.rviz -t cerise26_gemeo_rviz:latest docker
```

`servo6dof:humble` é o gerador (ROS 2, Fortress, `ros-humble-ign-ros2-control` e bibliotecas EGL) e define

`NVIDIA_DRIVER_CAPABILITIES=graphics,compute,utility,display` e `__EGL_VENDOR_LIBRARY_FILENAMES=/usr/share/glvnd/egl_vendor.d/10_nvidia.json`. `cerise26_gemeo` acrescenta MoveIt 2, bibliotecas de análise e FastAPI; `cerise26_gemeo_playwright` dirige o front-end em Chromium headless; `cerise26_gemeo_rviz` roda o RViz 2 numa tela virtual (Xvfb).

2. Em x86_64, troque a arquitetura do repositório apt da OSRF antes do primeiro build. As imagens base (`ros:humble-ros-base`, Playwright e NGC PyTorch) também existem para x86_64, mas o conjunto não foi testado nessa arquitetura:

```
sed -i 's/arch=arm64/arch=amd64/' docker/Dockerfile.spark
```

3. Confirme que o NVIDIA Container Toolkit injeta o vendor EGL da NVIDIA (deve aparecer `10_nvidia.json`):

```
sudo docker run --rm --gpus all servo6dof:humble ls /usr/share/glvnd/egl_vendor.d/
```

5.5. Etapa 2: geração do dataset

O gerador lê um plano fixo de poses, sobe N Gazebos isolados (worker w com `ROS_DOMAIN_ID` 90+w e `IGN_PARTITION` próprio, responsável pelas poses com `id % N == w`) e grava seis PNGs e um JSON por pose. A cada rodada, recalcula o que falta no disco e repete, o que torna a geração retomável. Faça primeiro um teste de 40 poses numa pasta separada. Antes da geração completa com `WS=~/braco`, renomeie o dataset baixado (`mv ~/braco/dataset ~/braco/dataset_release`) ou use outra pasta `WS`: o `gerar_hd_spark.sh` pula toda pose que já tem `NNNNNN.json` em `/ws/dataset` e, com o dataset do Release no lugar, termina na hora com `=== COMPLETO`, sem gerar nada.

1. Monte a pasta de trabalho (vista como `/ws` no contêiner):

```
R=~/cerise2026/gemeo/repo
WS=~/braco_teste                    # geração completa: WS=~/braco
mkdir -p $WS
cp -r $R/ros2_ws/src $WS/src
cp $R/dataset_tools/cams_hd.yaml $R/dataset_tools/gerar_hd_spark.sh $WS/
python3 $R/dataset_tools/gerar_plano.py --poses 40 --saida $WS/plano_poses.json
```

Na geração completa, omita `--poses` (5.000 poses, semente 20260909) e confira o plano: `cmp $WS/plano_poses.json $R/dataset_tools/plano_poses.json` não deve imprimir nada. Para a variante laranja, copie `dataset_tools/variante_laranja/servo6dof.urdf.xacro` sobre `$WS/src/servo6dof/servo6dof_description/urdf/servo6dof.urdf.xacro` antes do passo 2.

2. Compile o workspace no contêiner:

```
sudo docker run --rm -v $WS:/ws servo6dof:humble bash -lc \
  "source /opt/ros/humble/setup.bash && cd /ws && \
  colcon build --packages-up-to servo6dof_camera"
```

3. Inicie a geração em segundo plano (2 workers no teste; 8 na geração completa):

```
sudo docker run -d --name braco_gen --gpus all -v $WS:/ws servo6dof:humble \
  bash -lc "bash /ws/gerar_hd_spark.sh 2 > /ws/gen_main.log 2>&1"
```

Cada worker sobe `estudio.launch.py` com `headless:=true hd:=true` (mundo `estudio_hd.sdf`, ogre2), espera `BOOT` segundos (padrão 50) e roda `tirar_fotos.py` com `use_sim_time:=true`, trajetória de `DURACAO` = 3,0 s e assentamento `ASSENT` = 0,5 s em tempo de simulação. Essas variáveis podem ser passadas com `-e` no `docker run`.

4. Acompanhe o progresso:

```
tail -f $WS/gen_main.log             # "faltam N poses" a cada rodada; "=== COMPLETO" no fim
python3 $R/dataset_tools/status_dataset.py --plano $WS/plano_poses.json --saida $WS/dataset
nvidia-smi                           # os processos do Gazebo devem aparecer na GPU
```

Ignore a sugestão `bash continuar.sh` e a estimativa de tempo do `status_dataset.py`, herdadas da versão em WSL. Para retomar uma geração interrompida, remova o contêiner (`sudo docker rm -f braco_gen`) e repita o passo 3: as poses que já têm `NNNNNN.json` são puladas.

5. No fim, devolva os arquivos ao seu usuário e reconstrua o `index.csv` da pasta (juntas comandadas, garra e lista de imagens por pose):

```
sudo docker wait braco_gen && sudo docker rm braco_gen
sudo chown -R "$(id -u):$(id -g)" $WS/dataset $WS/gen_logs
python3 $R/dataset_tools/build_index.py $WS/dataset   # "index.csv reconstruido: N poses"
```

5.6. Etapa 3: índice, integridade e estatísticas

Execute a análise sobre `~/braco/dataset`, montado como somente leitura em `/dataset`; os metadados vão para `~/cerise2026/gemeo/dataset_meta`:

```
cd ~/cerise2026/gemeo/repo
bash dataset_tools/analise/rodar_analise.sh 0
bash scripts/rodar_container.sh rvc \
  "python3 dataset_tools/analise/real_vs_comandado.py --meta /dataset_meta"
```

O argumento `0` faz o `integridade.py` decodificar as 30.000 imagens. O pipeline roda `indexar.py` (índice com juntas comandadas e reais, split e arquivos), `integridade.py`, `estatisticas.py`, `grades.py` (grade de amostras e amostra versionada) e `linha_tempo.py` (lê `~/braco/gen_main.log` e `~/braco/gen_logs/`, se existirem). Esta e as etapas seguintes regravam as métricas e figuras de referência em `resultados/`, `evidencias/` e `amostra_dataset/`: compare com `git diff --stat` e restaure com `git checkout -- .`.

5.7. Etapa 4: modelo digital

1. Gere o URDF a partir do xacro; o script também compila `ros2_ws/`, cujo `install/` é usado pelas etapas seguintes:

```
cd ~/cerise2026/gemeo/repo
sudo docker run --rm -v $PWD:/repo cerise26_gemeo:latest \
  bash /repo/modelo_digital/gerar_urdf.sh
```

2. Produza as tabelas, a validação da cinemática direta (FK própria em numpy contra o TF do `robot_state_publisher`), o espaço de trabalho, os renders, o IoU entre a silhueta do modelo e a máscara do Gazebo e o MoveIt 2 com hardware falso:

```
bash scripts/rodar_container.sh tab "python3 modelo_digital/tabela_modelo.py"
bash scripts/rodar_container.sh fk "python3 modelo_digital/validar_fk_tf.py --n 300"
bash scripts/rodar_container.sh ws "python3 modelo_digital/espaco_trabalho.py --n 100000"
bash scripts/rodar_container.sh rend "python3 modelo_digital/render_modelo.py --ids 1 2 7"
bash scripts/rodar_container.sh iou "python3 modelo_digital/validar_projecao.py --n 1000"
bash scripts/rodar_container.sh moveit \
  "bash modelo_digital/moveit_headless/rodar_moveit_headless.sh"
```

Com `--n 5000`, o `validar_projecao.py` lista todas as imagens com rótulo suspeito.

5.8. Etapa 5: gêmeo digital com a verdade-base

1. Suba o servidor com o preditor oráculo (GT) e confira a saúde:

```
cd ~/cerise2026/gemeo/repo
bash gemeo_digital/subir_servidor.sh gt
curl -s http://127.0.0.1:8091/saude
```

O servidor roda no contêiner `cerise26_gemeo`, publicado só em `127.0.0.1:8091`, e transmite por WebSocket 150 poses do split de teste. Abra `http://127.0.0.1:8091` no navegador da mesma máquina; de outra máquina, use um túnel SSH local (`ssh -N -L 127.0.0.1:18091:localhost:8091 <usuario>@<servidor>`) e abra `http://127.0.0.1:18091`. O modo `ruido` (`RUIDO_GRAUS=4 bash gemeo_digital/subir_servidor.sh ruido`) soma ruído gaussiano à verdade-base só para exercitar o painel de erro.

2. Gere as capturas verificadas por Playwright (GT, ruído e, por fim, servidor em modo `auto`):

```
bash gemeo_digital/capturas/sessao_capturas.sh
```

3. Teste o nó ROS 2 sem Gazebo (imagens do dataset → `gemeo_twin_node` → `/twin/joint_states` → TF com prefixo `twin/`) e derrube o servidor:

```
bash scripts/rodar_container.sh ros2twin "bash gemeo_digital/ros2/testar_no_ros2.sh"
bash gemeo_digital/derrubar_servidor.sh
```

5.9. Etapa 6: gêmeo digital com o modelo de pose

No modo `pose`, o servidor carrega `inferencia/predict.py` (classe `PosePredictor`) do repositório `cerise-2026-pose-servo6dof`, montado de `~/cerise2026/pose`, e o gêmeo passa a seguir as juntas previstas só a partir da imagem.

1. Prepare a pasta do modelo de pose: modelo final das Releases, `READY.json` (sinaliza que o modelo está pronto), `index.npz` e `crop.json` do

conjunto de teste em que o modelo foi avaliado (500 poses, semente 2026), imagem Docker e pesos do Depth Anything V2. Detalhes em `inferencia/INFERENCIA.md` daquele repositório.

```
gh repo clone cerise-ufg/cerise-2026-pose-servo6dof ~/cerise2026/pose
cd ~/cerise2026/pose
gh release download artefatos-2026-10-26 -R cerise-ufg/cerise-2026-pose-servo6dof \
  -D _releases -p pose_servo6dof_modelo_final.tar -p SHA256SUMS.txt
(cd _releases && sha256sum -c --ignore-missing SHA256SUMS.txt)
tar -xf _releases/pose_servo6dof_modelo_final.tar  # -> inferencia/modelo/pose_servo6dof.pt
cp resultados/READY.json READY.json
mkdir -p dados/final
cp resultados/dados_convertidos/final/index.npz \
  resultados/dados_convertidos/final/crop.json dados/final/
sudo docker build -t cerise26_pose:latest -f docker/Dockerfile docker/
mkdir -p ~/cerise2026/hf_cache
sudo docker run --rm -v ~/cerise2026/pose:/workspace/pose \
  -v ~/cerise2026/hf_cache:/root/.cache/huggingface \
  cerise26_pose:latest python /workspace/pose/scripts/baixar_pesos_da.py vitl
```

Se você já seguiu o roteiro do modelo de pose com o clone em `~/cerise2026/pose`, pule o passo 1: o repositório, os Releases extraídos, a pasta `dados/final/` da sua conversão, a imagem `cerise26_pose:latest` e o cache `~/cerise2026/hf_cache` já existem. Falta só o `READY.json`: copie `resultados/READY.json` para usar o modelo final do Release ou, para usar o seu modelo, copie o pacote exportado para `inferencia/modelo/pose_servo6dof.pt` e gere o arquivo a partir da sua avaliação:

```
cd ~/cerise2026/pose
cp runs/pacote/pose_servo6dof.pt inferencia/modelo/pose_servo6dof.pt
python3 scripts/escrever_ready.py --saida READY.json \
  --metricas runs/repro_full_transfer/aval/metricas_teste.json
```

2. Suba o servidor na imagem do modelo de pose, com GPU, todas as poses do teste e gravação das medições por quadro:

```
cd ~/cerise2026/gemeo/repo
IMAGEM=cerise26_pose:latest GPU=1 N_QUADROS=0 LOGS=$HOME/cerise2026/gemeo/integracao/logs \
  bash gemeo_digital/subir_servidor.sh pose
```

3. Avalie o teste inteiro nas duas câmeras em malha fechada (o próximo quadro só sai depois que o front-end desenha o anterior) e com taxas-alvo de 2, 5 e 10 quadros/s; depois, analise e gere as capturas:

```
I=$HOME/cerise2026/gemeo/integracao
GL=gpu bash gemeo_digital/integracao/rodar_playwright.sh avaliar --fps_fixos 2 5 10 \
  --saida /integ/resultados/cliente_avaliacao_gpu.json
mkdir -p $I/logs/gpu && mv $I/logs/sessao_*.jsonl $I/logs/gpu/
EXTRA="-v $I:/integ -v $HOME/cerise2026/pose:/pose:ro" \
  bash scripts/rodar_container.sh analise \
  "python3 gemeo_digital/integracao/analisar_gemeo.py"
GL=gpu bash gemeo_digital/integracao/rodar_playwright.sh capturas
```

As saídas ficam em `~/cerise2026/gemeo/integracao/resultados` e `evidencias`.

4. Com o servidor ainda no ar, rode o nó ROS 2 com o modelo de pose e o RViz numa tela virtual (o nó envia cada imagem para `POST /api/prever`):

```
bash gemeo_digital/ros2/rodar_rviz_pose.sh
```

Saídas: `$I/resultados/ros2_twin_pose.json` e `$I/evidencias/a12i_rviz_gemeo_1.png` a `_3.png`.

5.10. Verificação do resultado

Contagens, erros e IoU devem coincidir com os valores abaixo quando se usam o mesmo dataset e o mesmo modelo; tempos, latências e FPS dependem da máquina.

Tabela 11. Valores de referência medidos em 26/10/2026 e arquivos onde estão

Verificação	O que você deve ver	Arquivo
Índice	5.000 poses, 30.000 PNGs, 0 órfãos, 0 faltantes; splits 4.000/500/500	`resultados/dataset_index_resumo.json`
Integridade	30.000 imagens decodificadas, 0 problemas, 0 quadros duplicados, todas 1280×720, rótulos {0, 1, 2} nas 10.000 máscaras (86 s com 6 processos)	`resultados/dataset_integridade.json`
Juntas reais × comandadas	34,1 % das poses divergem mais de 0,01 rad	`resultados/dataset_real_vs_comandado.json`
Geração completa	3,64 h com 8 workers; mediana de 12,8 s por pose por worker	`resultados/dataset_geracao.json`
URDF	13 links, 12 juntas (1 fixa, 11 de revolução), 5 juntas mimic	`resultados/gerar_urdf.log`
FK × TF	300 configurações × 11 links: erro máximo de 1,9e-15 m e 4,2e-8 rad	`resultados/validacao_fk_tf.json`
IoU modelo × imagens (1.000 poses)	0,950 (frontal) e 0,938 (diagonal)	`resultados/validacao_projecao_n1000.json`
MoveIt 2 headless	5/5 objetivos com SUCCESS e 3 controladores ativos	`resultados/moveit_cliente.log`
Servidor e nó ROS 2 (GT)	`"ok": true` com 150 quadros; erro do efetuador do gêmeo de 4,6e-16 m	`resultados/servidor_print.log`, `resultados/ros2_twin_teste.log`
Servidor (modo pose)	500 quadros, preditor `PosePredictor (HPE + Depth Anything V2, frente POSE)`, split do `index.npz`	`resultados/a12_integracao/terminal/servidor.txt`
Gêmeo com o modelo de pose	1.000 quadros; erro médio de 5,41° nas juntas 1–4 e de 12,68° nas 5 juntas; efetuador com mediana de 76 mm	`resultados/a12_integracao/metricas_gemeo_integracao.json`
Latência e FPS	ponta a ponta com mediana de 99,6 ms; 9,8 quadros/s (frontal, malha fechada); taxas-alvo 2/5/10 mantidas	`resultados/a12_integracao/terminal/avaliacao.txt`
ROS 2 + RViz (pose)	`/twin/joint_states` a 2,00 Hz, 79 de 80 pares casados, atraso mediano de 255 ms	`resultados/a12_integracao/ros2_twin_pose.json`

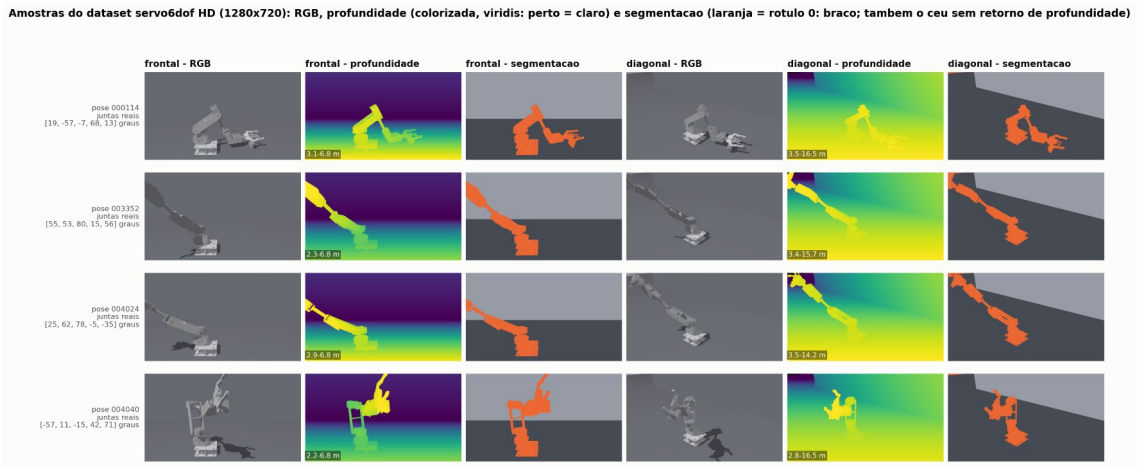


Figura 7. Amostras do dataset nas duas câmeras: RGB, profundidade colorizada e segmentação, em que o laranja marca o rótulo 0 (braço)

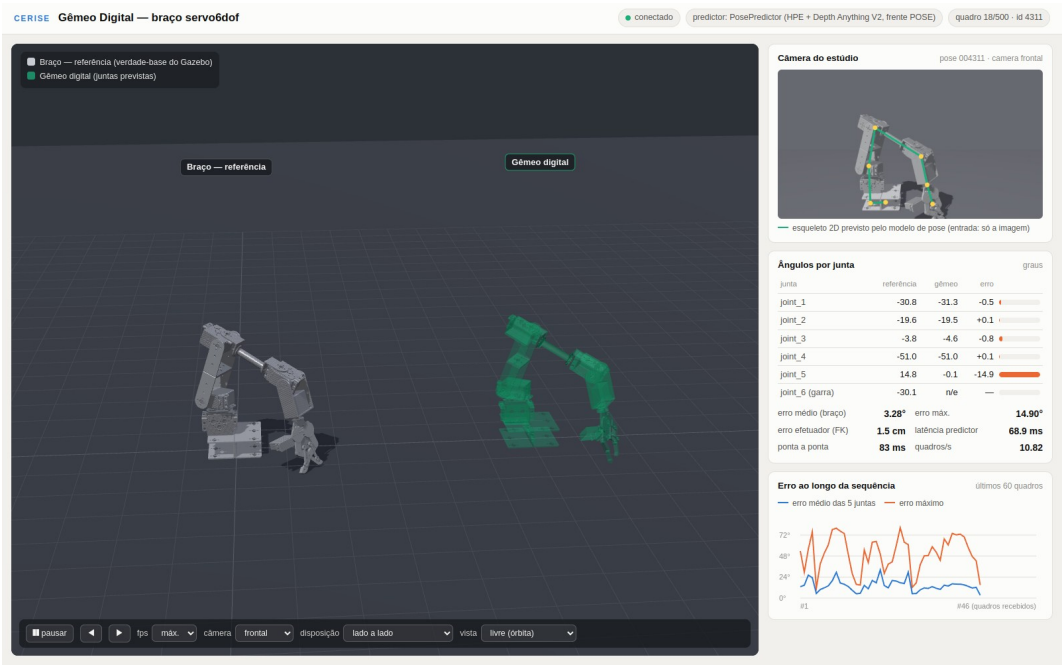


Figura 8. Front-end do gêmeo digital no modo pose: braço de referência e gêmeo lado a lado, imagem da câmera com o esqueleto previsto, ângulos por junta e gráfico de erro

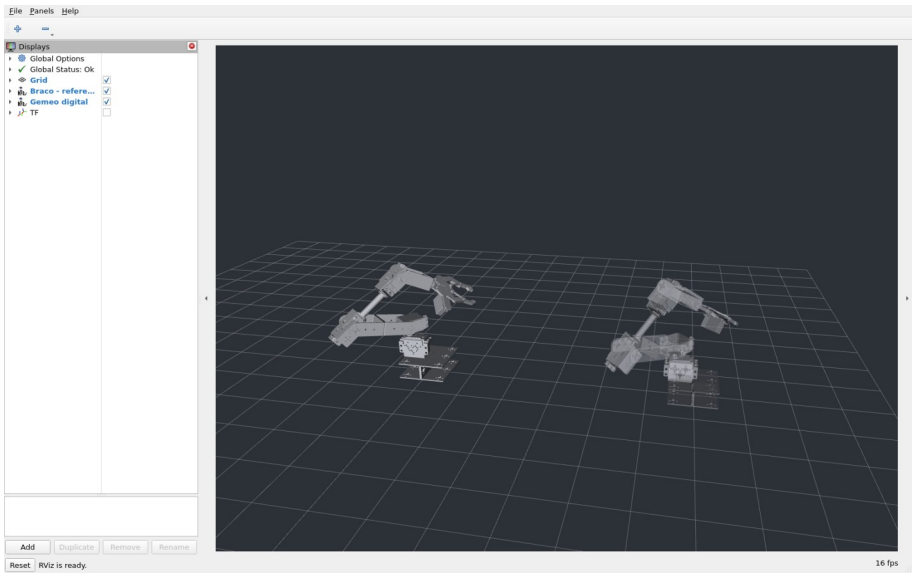


Figura 9. RViz 2 em tela virtual com o braço de referência e o gêmeo digital, que segue o tópico /twin/joint_states publicado pelo nó ROS 2

5.11. Problemas comuns e soluções

- **Os controladores não sobem e o braço não se move.** O xacro carrega o plugin `ign_ros2_control-system` (`servo6dof_gazebo.xacro`); sem o pacote `ros-humble-ign-ros2-control`, os spawners não encontram o `controller_manager`. O pacote já está no `Dockerfile.spark`; instale-o se montar outro ambiente.
- **A renderização cai no Mesa (software) ou o ogre2 aborta.** Rode com `--gpus all`, mantenha `graphics` em `NVIDIA_DRIVER_CAPABILITIES` e force `__EGL_VENDOR_LIBRARY_FILENAMES=/usr/share/glvnd/egl_vendor.d/10_nvidia.json`. Mantenha também `RENDER_DRIVER_OVERRIDE=""`: sem ela, o `estudio.launch.py` define `MESA_LOADER_DRIVER_OVERRIDE=zink`, configuração da antiga máquina WSL.
- **Não adicione o plugin Label ao URDF.** O plugin `Label` quebra os controladores (os `joint_states` param de ser publicados). Os rótulos ficam no `estudio_hd.sdf` (chão = 1, fundo = 2) e o braço sai com rótulo 0, o mesmo do céu sem retorno na câmera diagonal: obtenha a máscara do braço como rótulo 0 com profundidade > 0.
- **Fotos tiradas antes de o braço chegar.** A espera de cada pose deve ser por tempo de simulação (`/clock`, função `_espera_sim` de `tirar_fotos.py`), não por tempo de relógio, porque o fator de tempo real cai com vários Gazebos. Não remova `use_sim_time:=true` do `gerar_hd_spark.sh`.

- **Rodadas sem progresso** (`AVISO: ronda sem progresso`, `ABORT: 3 rondas sem progresso`). Consulte `gen_logs/w*.log`, aumente `BOOT` (por exemplo, `-e BOOT=90`) ou reduza os workers.
- **Rótulo errado no treino.** Use `juntas_reais`, não `juntas_comandadas` (34,1 % das poses divergem, sobretudo por contato com o chão), e descarte as imagens com IoU abaixo de 0,8 listadas em `resultados/validacao_projecao_n5000.csv` (2,47 %).
- **No modo auto, o gêmeo mostra erro zero.** O servidor caiu no oráculo por não achar `~/cerise2026/pose/READY.json`; veja `aviso_predictor` em `curl -s http://127.0.0.1:8091/api/info`. O modo `pose` depende do repositório `cerise-2026-pose-servo6dof`, da imagem `cerise26_pose:latest`, de `GPU=1` e do cache `~/cerise2026/hf_cache`; na subida, o servidor instala FastAPI e uvicorn por `pip` se faltarem (precisa de rede).
- **Playwright falha com GL=gpu.** Sem EGL da NVIDIA para o Chromium, use `GL=swiftshader` (renderização na CPU, cerca de 0,42 quadro/s).
- **Arquivos pertencem ao root.** Os contêineres gravam como root; use `sudo chown -R "$(id -u):$(id -g)"` na pasta.
- **Acesso externo.** Não publique a porta 8091 fora do loopback; use o túnel SSH da Etapa 5.

5.12. Referências do repositório

- `README.md` e `RELEASES.md`: visão geral, resultados, artefatos das Releases e SHA-256;
- `docs/DATASET.md`: ficha do dataset (formato, splits, limitações);
- `docker/Dockerfile.spark`, `docker/Dockerfile.gemeo`, `docker/Dockerfile.playwright` e `docker/Dockerfile.rviz`;
- `dataset_tools/gerar_plano.py`, `gerar_hd_spark.sh`, `cams_hd.yaml`, `build_index.py`, `status_dataset.py`, `analise/` e `variante_laranja/LEIAME.md`;
- `ros2_ws/src/servo6dof/servo6dof_camera/`: `launch/estudio.launch.py`, `scripts/tirar_fotos.py` e `worlds/estudio_hd.sdf`;
- `modelo_digital/`: `cinematica.py`, `gerar_urdf.sh`, `validar_fk_tf.py`, `validar_projecao.py` e `moveit_headless/`;
- `gemeo_digital/`: `servidor/app.py`, `servidor/predictors.py`, `subir_servidor.sh`, `integracao/avaliar_gemeo.py`, `integracao/analisar_gemeo.py`,

`integracao/sessao_2026-10-26/LEIAME.md` e `ros2/DESENHO_NO_ROS2.md`;

- `RELATORIO_SECAO.md` e `RELATORIO_SECAO_A12.md`: métodos e resultados completos.

6. TREINO E AVALIAÇÃO DO MODELO DE ESTIMAÇÃO DE POSE

6.1. O que será reproduzido

Este roteiro descreve como adaptar o modelo de estimação de pose de braço robótico desenvolvido em 2025 (HPE/HoRoPose com ramo de profundidade Depth Anything V2, variante MyNet) ao braço servo6dof, treiná-lo no dataset sintético de 2026 e avaliá-lo. O código está no repositório `cerise-2026-pose-servo6dof` da organização cerise-ufg no GitHub (privado; o acesso é concedido mediante solicitação ao bolsista). O roteiro cobre o Objetivo 3, Atividades 8 (aplicação do modelo de 2025 ao novo dataset) e 9 (avaliação de precisão e robustez), e a interface com o Objetivo 4: o módulo de inferência consumido pelo gêmeo digital na Atividade 12.

Ao final, você terá:

- a imagem Docker de treino `cerise26_pose:latest` (base NGC PyTorch 26.05);
- o dataset convertido para o formato do modelo (recortes 256×256 e profundidade relativa do Depth Anything V2);
- os modelos treinados por transferência dos pesos de 2025 (etapa 1, DepthNet; etapa 2, modelo completo) e pela linha de base ImageNet;
- as métricas no conjunto de teste: MAE por junta, erro 3D, ADD/AUC, PCK 2D, robustez a sete perturbações, generalização entre câmeras e latência;
- o pacote de inferência `pose_servo6dof.pt` e a classe `PosePredictor` funcionando sobre uma imagem do teste.

6.2. Requisitos

- Hardware: GPU NVIDIA com pelo menos 12 GB e 32 GB de RAM para o container (limite de `scripts/spark/rodar.sh`). Picos medidos de GPU: 9,3 GB no treino completo com lote 32 (`resultados/runs/final_full_transfer/treino.log`), 7,6 GB na profundidade (`resultados/logs/conversao_final.txt`) e 2,94 GB na inferência (`resultados/latencia_predictor.json`). O projeto usou um

NVIDIA DGX Spark (GPU GB10 Blackwell, sm_121, CPU aarch64, 128 GB de memória unificada).

- Sistema: Linux (o projeto usou Ubuntu 24.04) com Docker, NVIDIA Container Toolkit e driver com suporte a CUDA 13 (`resultados/logs/check_env.txt` registra o driver 580 em modo de compatibilidade).
- Software: git, GitHub CLI (`gh`), `sha256sum` e `tar` no hospedeiro. Dentro da imagem: Python 3.12.3, PyTorch 2.12, torchvision 0.27, numpy 2.1 (da base `nvcr.io/nvidia/pytorch:26.05-py3`) e, instalados pelo `docker/Dockerfile`, `opencv-python-headless==4.11.0.86`, `lxml`, `trimesh`, `easydict` e `six`.
- Contas e acessos: conta no GitHub com acesso aos repositórios `cerise-2026-pose-servo6dof` e `cerise-2026-servo6dof-gemeo-digital` (este último fornece o dataset); acesso à internet para baixar a imagem NGC, o Depth Anything V2 Large do Hugging Face e a ResNet-50 do torchvision. Nenhum token do Hugging Face é necessário; o `gh` exige apenas o seu login no GitHub.
- Espaço em disco: cerca de 60 GB livres (imagem Docker de 35,5 GB; dataset de 2,6 GB compactado e 2,5 GB extraído; dataset convertido de 2,6 GB; Releases de até 4,9 GB; cerca de 0,9 GB por rodada de treino completo).

Tabela 12. Tempos medidos no GB10 compartilhado com outros serviços (fonte: logs em `resultados/logs/` e `resultados/runs/`)

Passo	Tempo medido
Conversão das imagens (6 processos)	23 s
Profundidade Depth Anything V2 (10.000 recortes)	cerca de 16 imagens/s, ou 10 min
Etapa 1, DepthNet, 5 épocas sobre 3.151 poses	10,5 min (transferência) e 19,2 min (ImageNet)
Etapa 2, modelo completo, transferência, 20 épocas	61,8 min
Etapa 2, linha de base ImageNet, 12 épocas	52,5 min
Avaliação no teste, robustez e latência	cerca de 25 min

6.3. Obtenção do código e dos dados

1. Autentique o GitHub CLI e clone o repositório em `~/cerise2026/pose`, caminho em que o gêmeo digital procura o modelo (Etapa 6 do roteiro do repositório `cerise-2026-servo6dof-gemeo-digital`):

```
gh auth login
gh repo clone cerise-ufg/cerise-2026-pose-servo6dof ~/cerise2026/pose
cd ~/cerise2026/pose
```

2. Baixe os artefatos do Release `artefatos-2026-10-26`, confira os SHA-256 e extraia na raiz do repositório. Cada `.tar` já traz os caminhos relativos

(`pesos/`, `runs/<rodada>/`, `inferencia/modelo/`). O mínimo para treinar é `pesos2025_realsense_full_checkpoint.tar`, que contém o checkpoint de 2025 usado na transferência. Os `ckpt_*.tar` trazem os checkpoints originais, para comparação, e `pose_servo6dof_modelo_final.tar` traz o modelo final para inferência:

```
gh release download artefatos-2026-10-26 -R cerise-ufg/cerise-2026-pose-servo6dof \
  -D _releases
cd _releases && sha256sum -c SHA256SUMS.txt && cd ..
for f in _releases/*.tar; do tar -xf "$f"; done
```

3. Baixe o dataset sintético (versão cinza, a usada em todos os resultados) do Release `dados-2026-10-26` do repositório do gêmeo digital. Extraia as duas partes na mesma pasta e confira arquivo por arquivo com o manifesto. Se você já o baixou pelo roteiro do gêmeo digital (pasta `~/braco/dataset`), pule este passo e use `DATASET="$HOME/braco/dataset"` na seção seguinte:

```
gh release download dados-2026-10-26 -R cerise-ufg/cerise-2026-servo6dof-gemeo-digital \
  -D "$HOME/dados_gemeo" -p 'dataset_cinza.*' -p SHA256SUMS.txt
cd "$HOME/dados_gemeo"
sha256sum -c --ignore-missing SHA256SUMS.txt
tar -xf dataset_cinza.part01.tar && tar -xf dataset_cinza.part02.tar
sha256sum -c --quiet dataset_cinza.manifest.sha256
cd -
```

A pasta `dataset_cinza/` deve ter 35.009 arquivos: 5.000 JSON, 30.000 PNG (RGB, profundidade e segmentação das câmeras frontal e diagonal) e os auxiliares. No Windows, use `Get-FileHash -Algorithm SHA256 <arquivo>` e o `tar` nativo.

6.4. Preparação do container de treino

1. Construa a imagem. O `docker/Dockerfile` parte de `nvcr.io/nvidia/pytorch:26.05-py3` e instala as dependências do código de 2025. Use `sudo` se o seu usuário não estiver no grupo `docker`:

```
docker build -t cerise26_pose:latest -f docker/Dockerfile docker/
```

2. Abra um shell no container. As montagens reproduzem as de `scripts/spark/rodar.sh`: o repositório em `/workspace/pose`, os dados convertidos em `/dados`, o dataset (somente leitura) em `/dados/dataset`, as rodadas em `/runs` e os caches do Hugging Face (em `~/cerise2026/hf_cache`, o mesmo do gêmeo digital e de `scripts/spark/rodar.sh`) e do torchvision no hospedeiro, para baixar os pesos uma única vez:

```
mkdir -p dados runs pesos "$HOME/cerise2026/hf_cache" "$HOME/.cache/torch"
export DATASET="$HOME/dados_gemeo/dataset_cinza"
docker run --rm -it --gpus all --ipc=host --memory=32g --memory-swap=32g \
  -e HPE_HRNET_W32=/workspace/pose/pesos/hrnet_w32-36af842e_roc.pth -e PYTHONUNBUFFERED=1 \
  -v "$HOME/cerise2026/hf_cache":/root/.cache/huggingface \
  -v "$HOME/.cache/torch":/root/.cache/torch \
  -v "$PWD":/workspace/pose -v "$PWD/dados":/dados -v "$DATASET":/dados/dataset:ro \
  -v "$PWD/runs":/runs -w /workspace/pose cerise26_pose:latest bash
```

3. Dentro do container, confira o ambiente. A linha `cuda True` deve mostrar a sua GPU; `timm MISSING` é esperado, porque o código não usa o `timm`:

```
python docker/check_env.py
```

Todos os comandos das seções seguintes rodam dentro desse shell, a partir de `/workspace/pose`. Para treinos longos, use `docker run -d --name <nome> ... cerise26_pose:latest python ...`, como faz `scripts/spark/rodar.sh`, e acompanhe com `docker logs -f <nome>`.

6.5. Pesos de terceiros e verificação da geometria

1. Baixe o Depth Anything V2 Large (arquivo `depth_anything_v2_vitl.pth` do repositório `depth-anything/Depth-Anything-V2-Large` do Hugging Face, licença CC-BY-NC-4.0). O script imprime o caminho do arquivo no cache:

```
python scripts/baixar_pesos_da.py vitl
```

2. Somente para a linha de base ImageNet, obtenha a HRNet-W32 pré-treinada no ImageNet: baixe o arquivo `hrnet_w32-36af842e.pth` pelo link do repositório oficial HRNet/HRNet-Image-Classification, salve-o como `pesos/hrnet_w32-36af842e_roc.pth` e compare o SHA-256 com o do arquivo usado no projeto, registrado no `README.md` (`12682d70…a4c6`). Ao treinar a DepthNet ImageNet, o terminal deve mostrar `Loading hrnet pretrained weights (ImageNet) from ...`; sem essa linha, a HRNet começa com pesos aleatórios. A ResNet-50 ImageNet é baixada sozinha pelo torchvision no primeiro treino.

3. Verifique a geometria. O script projeta os keypoints da cinemática direta do URDF versionado (`servo6dof/servo6dof.urdf`) sobre as imagens e mede quantos caem sobre a máscara do robô:

```
python scripts/verificar_geometria.py --dataset /dados/dataset --saida /runs/geometria
```

6.6. Conversão do dataset

1. Converta o dataset. O conversor calcula os keypoints 2D e 3D, a matriz K e os ângulos `juntas_reais`, divide as poses em 80/10/10 por id (semente 2026) e grava os recortes 256×256. Passe sempre o `crop.json` da execução original: ele fixa os recortes por câmera, que fazem parte da entrada do modelo:

```
python scripts/converter_dataset.py --dataset /dados/dataset --saida /dados/final \
  --workers 6 --crop_json resultados/dados_convertidos/final/crop.json
```

2. Calcule a profundidade relativa do Depth Anything V2 para cada recorte (gera `depth256_{treino,val,teste}.npy`):

```
python scripts/precomputar_profundidade.py --dados /dados/final --bs 16
```

6.7. Treino com transferência e linha de base

O `scripts/treinar.py` implementa as duas etapas de 2025. Na etapa 1, a DepthNet aprende a profundidade da raiz (perda L1). Na etapa 2, o modelo

completo (ResNet-50, camada integral, DepthNet da etapa 1, regressão iterativa das juntas e cinemática direta) é treinado com as perdas do estágio sintético de 2025. Com `--init 2025`, os tensores do checkpoint de 2025 cujas formas casam são carregados. Com `--init imagenet`, só os backbones vêm do ImageNet. Os argumentos abaixo se baseiam nos registrados no cabeçalho (`=== ... args={...}`) de cada `resultados/runs/<rodada>/treino.log`. A execução original também limitou o tempo com `--tempo_max_min` (26 min na etapa 1 por transferência, 70 min na etapa 2 por transferência e 30 min no experimento entre câmeras, limites que não chegaram a ser atingidos) e usou a conversão `parcial2` na etapa 1 e no experimento entre câmeras (ver Problemas comuns). Use pastas `--saida` novas: o script acrescenta linhas a `treino.log` e `curvas.csv` já existentes.

1. Treine a etapa 1 (DepthNet) com os pesos de 2025 e com ImageNet:

```
python scripts/treinar.py --dados /dados/final --saida /runs/repro_etapa1_transfer \
  --etapa depthnet --init 2025 \
  --ckpt_2025 /workspace/pose/pesos/realsense_full_checkpoint.pk \
  --epocas 5 --inicio_decaimento 2 --workers 3
python scripts/treinar.py --dados /dados/final --saida /runs/repro_etapa1_imagenet \
  --etapa depthnet --init imagenet --epocas 5 --inicio_decaimento 2 --workers 3
```

2. Treine a etapa 2 por transferência (20 épocas) e a linha de base. Na execução original, a linha de base foi lançada com `--epocas 20 --tempo_max_min 50` e parou na 12ª época pelo limite de tempo (`resultados/runs/final_full_imagenet/treino.log`, que registra também uma primeira tentativa com 45 min, reiniciada um minuto depois); o comando abaixo fixa as 12 épocas, e você pode usar 20 para comparar no mesmo número de épocas:

```
python scripts/treinar.py --dados /dados/final --saida /runs/repro_full_transfer \
  --etapa full --init 2025 --rootnet /runs/repro_etapa1_transfer/melhor.pt \
  --epocas 20 --bs 32 --workers 4
python scripts/treinar.py --dados /dados/final --saida /runs/repro_full_imagenet \
  --etapa full --init imagenet --rootnet /runs/repro_etapa1_imagenet/melhor.pt \
  --epocas 12 --bs 32 --workers 4
```

3. Para o experimento de generalização entre câmeras, treine só com a câmera frontal:

```
python scripts/treinar.py --dados /dados/final --saida /runs/repro_xcam_frontal \
  --etapa full --init 2025 --rootnet /runs/repro_etapa1_transfer/melhor.pt \
  --epocas 10 --bs 32 --workers 3 --cams_treino frontal --cams_val frontal
```

Cada rodada grava `melhor.pt` (melhor época na validação), `ultimo.pt`, `curvas.csv`, `treino.log` e, na transferência, `transfer_info.json`. Para continuar um treino interrompido, use `--retomar <ultimo.pt> --epoca_inicial <épocas feitas>`.

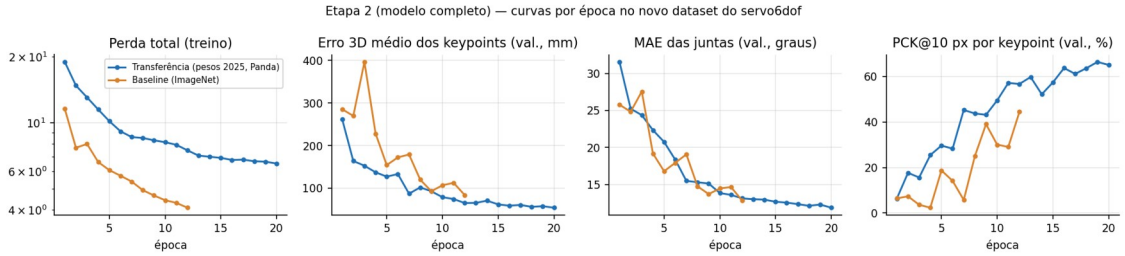


Figura 10. Curvas por época da etapa 2: perda de treino e métricas de validação da transferência e da linha de base (resultado esperado)

6.8. Avaliação no conjunto de teste

1. Avalie os três modelos nas 1.000 imagens do teste (500 poses × 2 câmeras). `--robustez` aplica as sete perturbações a um subconjunto fixo de 400 imagens, recalculando a profundidade sobre a imagem perturbada, e `--latencia` mede o Depth Anything V2 e o HPE com lote 1:

```
python scripts/avaliar.py --dados /dados/final --ckpt /runs/repro_full_transfer/melhor.pt \
  --saida /runs/repro_full_transfer/aval --robustez --n_robustez 400 --latencia
python scripts/avaliar.py --dados /dados/final --ckpt /runs/repro_full_imagenet/melhor.pt \
  --saida /runs/repro_full_imagenet/aval --robustez --n_robustez 400
python scripts/avaliar.py --dados /dados/final --ckpt /runs/repro_xcam_frontal/melhor.pt \
  --saida /runs/repro_xcam_frontal/aval
```

Cada avaliação grava `metricas_teste.json` (todas as câmeras e cada câmera separadamente) e `predicoes_teste.npz`. Para conferir os checkpoints originais sem treinar, troque `--ckpt` por `/runs/final_full_transfer/melhor.pt` (ou `final_full_imagenet`, `xcam_frontal_transfer`) e use outra pasta `--saida`.

2. Gere as tabelas no formato do relatório. Com `resultados` como argumento, o script reproduz `resultados/tabelas_relatorio.md`. Para as suas rodadas, copie os JSON com os nomes esperados:

```
mkdir -p /runs/minhas_tabelas
cp /runs/repro_full_transfer/aval/metricas_teste.json \
  /runs/minhas_tabelas/metricas_teste_transfer.json
cp /runs/repro_full_imagenet/aval/metricas_teste.json \
  /runs/minhas_tabelas/metricas_teste_imagenet.json
cp /runs/repro_xcam_frontal/aval/metricas_teste.json \
  /runs/minhas_tabelas/metricas_teste_xcam.json
python scripts/tabelas.py /runs/minhas_tabelas
```

O `scripts/figuras.py --res <pasta> --out <pasta>` gera as figuras, desde que a pasta siga a mesma convenção de nomes de `resultados/` (`curvas_full_transfer.csv`, `predicoes_teste_transfer.npz` etc.).

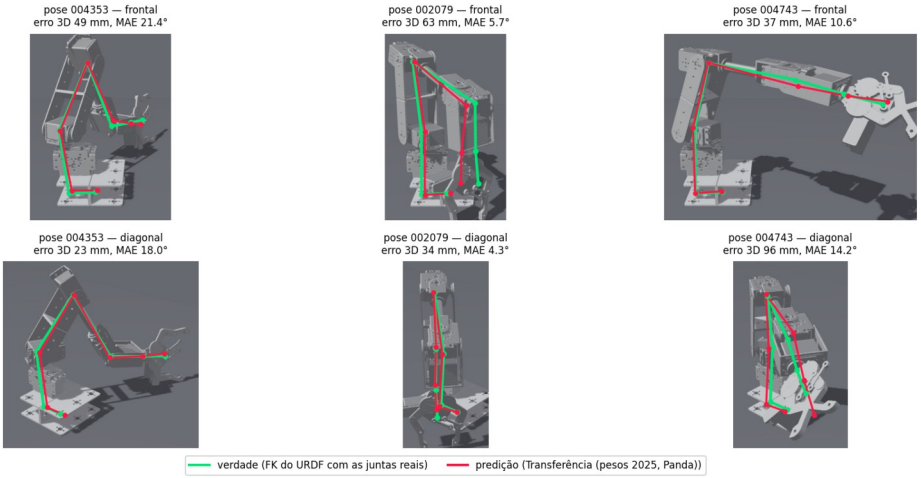


Figura 11. Predição do modelo de transferência (vermelho) sobre a verdade (verde) em três poses do teste, nas duas câmeras

6.9. Exportação e módulo de inferência

1. Empacote o checkpoint para inferência. O pacote reúne pesos, hiperparâmetros, recortes, intrínsecos e métricas de teste:

```
python scripts/exportar_modelo.py --ckpt /runs/repro_full_transfer/melhor.pt \
  --dados /dados/final --saida /runs/pacote/pose_servo6dof.pt \
  --metricas /runs/repro_full_transfer/aval/metricas_teste.json
```

2. Teste o `PosePredictor` pela linha de comando com uma imagem do teste versionada no repositório. O comando usa o modelo final do Release; para testar o seu pacote, troque `--ckpt` (e o primeiro argumento do `latencia_predictor.py`) por `/runs/pacote/pose_servo6dof.pt`:

```
python inferencia/predict.py --ckpt inferencia/modelo/pose_servo6dof.pt \
  --imagem resultados/imagens_teste/002079_camera_frontal_image.png --camera frontal \
  --saida /runs/overlay_002079.png --repeticoes 30
python scripts/latencia_predictor.py inferencia/modelo/pose_servo6dof.pt \
  /dados/dataset /runs/latencia.json
```

3. Use a classe em Python. A entrada é uma imagem RGB (não BGR) 1280×720 de uma das câmeras fixas do estúdio:

```
import sys; sys.path.insert(0, "/workspace/pose/inferencia")
import cv2
from predict import PosePredictor
p = PosePredictor("/workspace/pose/inferencia/modelo/pose_servo6dof.pt", device="cuda")
img = "resultados/imagens_teste/002079_camera_frontal_image.png"
rgb = cv2.imread(img)[..., ::-1].copy()
out = p.predict(rgb, camera="frontal")
print(out["joints"], out["latency_ms"])  # joint_1..joint_5 em rad; keypoints_2d/3d também
```

Para só usar o modelo, sem treinar, bastam o `pose_servo6dof_modelo_final.tar`, o Depth Anything V2 e a imagem Docker: retire a montagem do dataset do `docker run` e siga `inferencia/INFERENCIA.md`. Depois do primeiro download, `-e HF_HUB_OFFLINE=1` evita acessos ao Hugging Face.

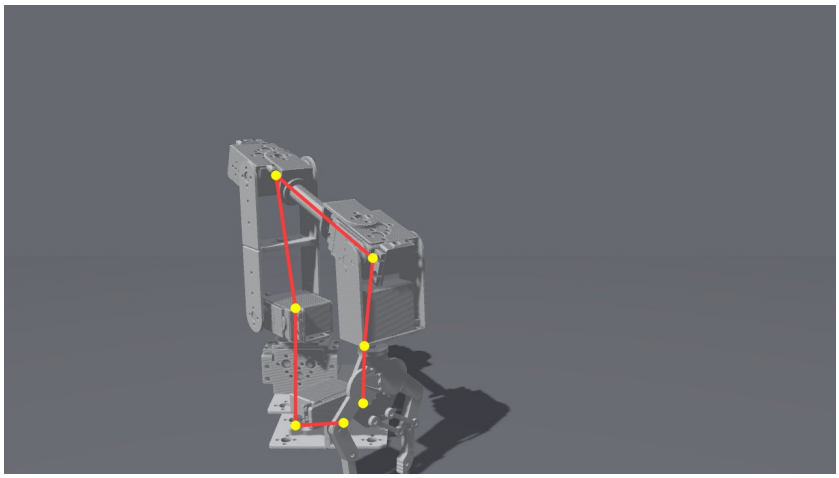


Figura 12. Saída esperada do predict.py: esqueleto previsto sobre a imagem 002079 da câmera frontal

6.10. Verificação do resultado

- Geometria: 99,9% (frontal) e 99,6% (diagonal) dos keypoints dentro do quadro caem sobre a máscara do robô (`resultados/verificacao_geometria.json`).
- Conversão: `ids completos: 5000`, recortes `frontal [-38.7, -162.5, 1045.0]` e `diagonal [122.5, -76.9, 846.6]`, e 8.000/1.000/1.000 imagens em treino, validação e teste (`resultados/logs/conversao_final.txt`).
- Transferência: o log mostra `2361/2366 tensores (98.1% dos parametros) carregados`; só `fc_pose_1.weight`, `decpose.weight` e `decpose.bias` mudam de forma (8 juntas do Panda para 5).
- Etapa 1: erro de profundidade da raiz de 71 mm na validação após 5 épocas (`resultados/runs/etapa1_depthnet_transfer_v2/curvas.csv`).
- Etapa 2, época 20: erro 3D de 54,1 mm e MAE de 11,86° na validação (`resultados/runs/final_full_transfer/curvas.csv`).
- Inferência na pose 002079 (frontal): `joints_graus` igual a -50,92; 1,0; -20,08; -78,53; 0,2, contra a verdade -43,23; 0,05; -17,5; -69,18; -7,69 (`resultados/logs/demo_predict.log`). `PosePredictor` com 162,5 ms de média (6,15 FPS) e 2,94 GB de GPU (`resultados/latencia_predictor.json`).

Tabela 13. Resultados de referência no teste (`resultados/metricas_teste_transfer.json`, `metricas_teste_imagenet.json` e `metricas_teste_xcam.json`)

Métrica	Transferência	Linha de base
MAE médio das 5 juntas	12,68°	13,51°
MAE por junta (1 a 5)	4,19 / 3,04 / 3,91 / 10,52 / 41,73°	7,20 / 3,87 / 5,41 / 9,25 / 41,83°
Erro 3D médio / mediano	58,7 / 41,1 mm	88,3 / 65,7 mm
Acurácia ADD a 60 mm / AUC-ADD 0–100 mm	68,0% / 0,471	32,8% / 0,265
PCK 2D a 5 / 10 / 20 px	30,1 / 61,7 / 87,1%	19,9 / 43,6 / 71,4%
Robustez: sem perturbação, ruído σ = 0,05, oclusão de 30%	57,4 / 99,0 / 94,3 mm	88,2 / 109,7 / 114,5 mm
Treino só com a frontal: erro 3D na frontal / diagonal	140,4 / 418,5 mm	—

Ao refazer as etapas com o dataset completo, espere valores próximos, mas não idênticos (ver a próxima seção). A avaliação dos checkpoints do Release só foi executada no GB10; em outra GPU, pequenas diferenças podem vir da profundidade recalculada.

6.11. Problemas comuns e soluções

- Código de 2025 com dois erros: na variante MyNet, a profundidade da raiz era calculada depois da camada integral, que recebia `root_trans = 0`; e `train_full.py` chamava `optimizer.step()` duas vezes por iteração. Os dois já estão corrigidos no repositório (`hpe2025/models/full_net.py`, trecho marcado `[porte 2026] CORRECAO`, e um único `opt.step()` em `scripts/treinar.py`). Não treine com o código de `legado_2025/`, mantido sem alterações como registro.
- Depth Anything V2 Large: vem do Hugging Face no primeiro uso. Sem internet no container, rode `scripts/baixar_pesos_da.py` numa máquina conectada e copie o cache para `$HOME/cerise2026/hf_cache`. O aviso de requisições não autenticadas pode ser ignorado.
- Latência maior que a de referência, ou instável: as medidas foram feitas com a GPU compartilhada com outros serviços. A decomposição de `avaliar.py --latencia` (213,8 ms: 138,9 do Depth Anything V2 e 74,9 do HPE) foi medida com um treino rodando em paralelo. Meça com a GPU livre. O Depth Anything V2 ViT-L responde por cerca de 65% do custo.
- MAE de cerca de 42° na junta 5: é esperado e não indica erro de execução. O keypoint da garra fica sobre o eixo da rotação da garra, então essa junta não é observável pelos keypoints atuais, e a `joint_6` não é estimada. Corrigir exige novos keypoints (pontas das pinças) e novo treino.

- Métricas ruins com os checkpoints do Release: confira se a conversão usou `--crop_json resultados/dados_convertidos/final/crop.json` e o dataset cinza, não a variante laranja.
- Falta de memória: reduza `--bs` e `--workers`. Não rode dois treinos completos ao mesmo tempo; na execução original, a linha de base em paralelo foi interrompida por falta de memória (`resultados/runs/final_full_imagenet_interrompido_memoria/`).
- Diferenças na etapa 1 e no experimento entre câmeras: na execução original, os dois usaram a conversão `parcial2`, com as 3.151 poses de treino disponíveis naquele momento (`resultados/dados_convertidos/parcial2/`). Para repetir exatamente a etapa 2, use `--rootnet /runs/etapa1_depthnet_transfer_v2/melhor.pt` (ou `etapa1_depthnet_imagenet_v2`), extraído de `ckpt_etapa1_depthnet_v2.tar`.
- Arquivos criados pelo container pertencem ao root: rode `sudo chown -R $(id -u):$(id -g) dados runs` no hospedeiro. Não é preciso rodar `servo6dof/gerar_urdf.sh` (usa caminhos da máquina original e a imagem ROS 2 do gêmeo): o URDF já está versionado.
- x86_64 com GPU NVIDIA comum: a imagem base NGC também é publicada para x86_64 e o código não depende de aarch64, mas essa combinação não foi testada. Em GPUs sem bfloat16 (anteriores à Ampere), desligue a precisão mista do treino com `--sem_amp` (também não testado).

6.12. Referências do repositório

- `README.md` e `RELEASES.md`: visão geral, comandos e artefatos com SHA-256; `RELATORIO_SECAO.md`: metodologia e discussão.
- `docker/Dockerfile` e `docker/check_env.py`: imagem e verificação do ambiente.
- `scripts/converter_dataset.py`, `scripts/precomputar_profundidade.py` e `scripts/verificar_geometria.py`: dados e geometria.
- `scripts/treinar.py`, `pose26/modelo.py` e `pose26/dados.py`: treino, modelo, transferência e augmentations.
- `scripts/avaliar.py`, `pose26/metricas.py`, `scripts/tabelas.py` e `scripts/figuras.py`: avaliação, tabelas e figuras.
- `inferencia/predict.py`, `inferencia/INFERENCIA.md` e `scripts/exportar_modelo.py`: módulo de inferência.

- `hpe2025/` (porte do código de 2025), `servo6dof/geometria.py` e `servo6dof/servo6dof.urdf` (cinemática e câmeras).
- `resultados/`: métricas, curvas, logs e metadados de todas as rodadas.

7. PROTÓTIPO DE COMUNICAÇÃO LORA

7.1. O que será reproduzido

Este roteiro reproduz o protótipo de comunicação LoRa entre um nó no drone (papel *node*) e uma estação de solo (papel *gateway*), da Atividade 10 do Objetivo 3, e o projeto do módulo de comunicação dos drones (Objetivo 5, itens 3 e 4, só a parte de projeto). O software foi verificado sem rádio, por compilação, testes, validação cruzada C × Python e simulação.

O ensaio físico entre duas placas ainda não foi realizado, porque as placas não foram adquiridas. As seções de montagem, gravação e ensaio descrevem o que deve ser feito quando elas chegarem; quem as executar fará a primeira medição real do enlace.

Ao final, você terá:

- o firmware compilado para os 5 ambientes (Heltec WiFi LoRa 32 V3 e TTGO LoRa32 v2.1, nó e gateway, e nó com GPS);
- os 30 testes nativos do protocolo `cerise_proto` e os 21 testes Python aprovados;
- as tabelas de tempo no ar e de alcance, a simulação de vários drones no canal e o dashboard alimentado por um fluxo serial simulado;
- os STL do gabinete do módulo e as estimativas de massa e consumo;
- a bancada de duas placas gravada e o ensaio de alcance e de taxa de entrega de pacotes (PDR), ainda inédito.

7.2. Requisitos

- **Computador:** PC x86_64 com Windows 10 ou 11 (usado: notebook com Windows 11). Não há dependência de GPU, de aarch64 nem da estação DGX Spark. Linux e macOS devem funcionar, mas não foram testados.
- **Software:** Git; GitHub CLI (`gh`), opcional; PlatformIO Core 6.1.19; Python 3.11 (usado 3.11.9); GCC para os testes nativos (usado WinLibs GCC 16.2.0, UCRT, r3).
- **Bibliotecas do firmware**, baixadas pelo PlatformIO nas versões de `firmware/platformio.ini`: `espressif32@6.12.0` (Arduino-ESP32 2.0.17), RadioLib 7.8.1, U8g2 2.36.18, TinyGPSPlus 1.1.0 e Unity 2.6.1.

- **Acesso:** conta no GitHub com acesso à organização `cerise-ufg` (repositório privado; solicite ao bolsista). O projeto não usa chaves nem senhas.
- **Disco:** cerca de 5 GB (pacotes do PlatformIO de 1,5 a 2,4 GB, `firmware/.pio` com 0,85 GB, WinLibs com 0,95 GB, ambiente Python com 0,4 GB).
- **Tempo:** 1 h para preparar e compilar pela primeira vez; 15 min para testes, ferramentas e dashboard; 1 a 2 h para a bancada; um dia de campo para o ensaio (estimativa).

7.3. Obtenção do código e dos dados

1. Clone o repositório e confira os binários versionados em `firmware/binarios/` (no Git Bash ou no Linux); as 15 linhas devem terminar em `OK`:

```
git clone https://github.com/cerise-ufg/cerise-2026-lora-prototipo.git
cd cerise-2026-lora-prototipo/firmware/binarios
sha256sum -c SHA256SUMS.txt
```

2. Opcionalmente, numa pasta fora do repositório, baixe o Release `v1.0-prototipo` (pelo `gh`, após `gh auth login`, ou pela página Releases). Ele traz só arquivos de depuração (`firmware.elf` e `firmware.map` dos 5 ambientes e o executável dos testes nativos), sem os quais tudo funciona:

```
gh release download v1.0-prototipo -R cerise-ufg/cerise-2026-lora-prototipo `
  -p "lora-firmware-v1-elf-map.zip"
(Get-FileHash .\lora-firmware-v1-elf-map.zip -Algorithm SHA256).Hash.ToLower()
# deve ser 2148810e3fe4d4a3e2e1458f68d98f5c1ba44dbb7bc376208c413578c45a7e17
Expand-Archive .\lora-firmware-v1-elf-map.zip -DestinationPath .\lora-firmware-v1
```

No Linux, use `sha256sum` e `unzip lora-firmware-v1-elf-map.zip -d lora-firmware-v1`; o `SHA256SUMS.txt` interno confere cada arquivo extraído.

7.4. Preparação do ambiente

1. Instale o PlatformIO Core 6.1.19 (usado: extensão PlatformIO IDE do VS Code, que cria `%USERPROFILE%\.platformio\penv\Scripts\pio.exe`; também há o pacote `platformio` do pip) e ponha-o no PATH:

```
$env:Path = "$env:USERPROFILE\.platformio\penv\Scripts;$env:Path"
pio --version      # PlatformIO Core, version 6.1.19
```

2. No Windows, instale o WinLibs (winlibs.com ou GitHub `brechtsanders/winlibs_mingw`) e ponha a pasta `mingw64\bin` no PATH; no Linux, use o GCC da distribuição (não testado):

```
$env:Path = "C:\caminho\para\mingw64\bin;$env:Path"   # ajuste ao local da extração
gcc --version      # ... built by Brecht Sanders, r3) 16.2.0
```

3. Na raiz do repositório, crie o ambiente Python (a pasta `.venv/` já está no `.gitignore`):

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r tools/requirements.txt
```

7.5. Compilação do firmware

1. Compile os 5 ambientes; a primeira execução baixa plataforma, compiladores e bibliotecas:

```
cd firmware
pio run
```

Tabela 14. Ambientes do firmware definidos em firmware/platformio.ini

Ambiente	Placa	Papel	Particularidade
`heltec_v3_node`	Heltec V3 (ESP32-S3 + SX1262)	nó	`NODE_ID=1`, UART1 para o computador de bordo
`heltec_v3_gateway`	Heltec V3	gateway	JSON na serial e OLED
`heltec_v3_node_gps`	Heltec V3 + GPS	nó	`-DUSE_GPS`
`ttgo_v21_node`	TTGO LoRa32 v2.1 (ESP32 + SX1276)	nó	`NODE_ID=2`
`ttgo_v21_gateway`	TTGO LoRa32 v2.1	gateway	placa alternativa

2. Para um só ambiente, use `pio run -e heltec_v3_gateway`. Os padrões de rádio ficam nos `build_flags` de `[esp32_base]`: `-DLORA_AU915_CH=8` (916,8 MHz), `-DLORA_SF=9`, `-DLORA_BW_KHZ=125.0`, `-DLORA_TX_DBM=14` e `-DCERISE_NET_ID=42`.

Cada ambiente deve terminar em `SUCCESS`, com as imagens em `firmware/.pio/build/<ambiente>/`. Com outras versões de compilador ou bibliotecas, os binários não serão idênticos bit a bit aos de `firmware/binarios/`.

7.6. Testes do protocolo

1. Com o GCC no PATH, rode em `firmware/` os 30 testes Unity da biblioteca `cerise_proto` (CRC-16, codificação dos nove tipos, erros de bit, ACK, duplicatas, tempo no ar, AU915 e JSON):

```
pio test -e native -v
```

2. Na raiz, rode os 21 testes Python (validação cruzada C × Python, compatibilidade com a frente de drones e dashboard). O teste `test_compat_with_drones_front` precisa do repositório `cerise-2026-drones-segmentacao` clonado ao lado; sem ele, é pulado e os outros 20 continuam válidos:

```
cd ..
$env:CERISE_DRONES_DIR = "..\cerise-2026-drones-segmentacao"
python -m pytest -v tools/tests
```

Com `gcc` no PATH, os testes recompilam `tools/c/gen_vectors.c` e regravam `tools/tests/vectors.tsv`; como o gerador é determinístico, `git status` não deve acusar mudança.

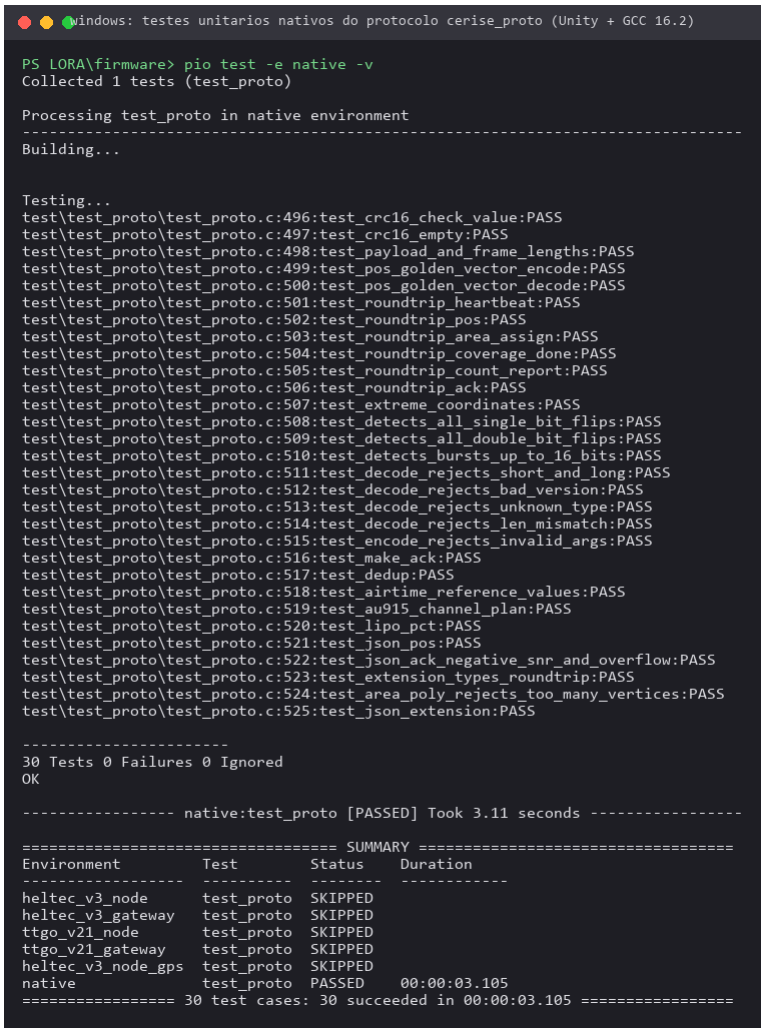


Figura 13. Saída esperada dos testes nativos do protocolo (30 testes, 0 falhas)

7.7. Ferramentas de análise

Por padrão, os scripts regravam `resultados/` e `evidencias/` (na máquina usada, com conteúdo idêntico); confira com `git status` e desfaça com `git restore`. `lora_calc.py` e `sim_canal.py` aceitam `--out-res` e `--out-fig`.

1. Calcule tempo no ar por mensagem e SF, maior quadro dentro de 400 ms, sensibilidade e alcance (Friis, log-distância e Okumura–Hata). O padrão é 14 dBm com o drone a 60 m; a segunda linha simula outro cenário fora do repositório:

```
python tools/lora_calc.py
python tools/lora_calc.py --ptx 20 --alt 120 --out-res ..\lora_saida --out-fig ..\lora_saida
```

2. Rode a simulação de N drones no mesmo canal (ALOHA, ALOHA + ACK, CSMA/CAD + ACK, slotted ALOHA e TDMA + ACK). A versão completa (1 130 execuções) leva cerca de 5 min; `--quick` levou 15 s, com números um pouco diferentes dos CSV versionados; `--plot-only` só refaz as figuras:

```
python tools/sim_canal.py --quick --out-res ..\lora_saida --out-fig ..\lora_saida
python tools/sim_canal.py
python tools/sim_canal.py --plot-only
python tools/fig_protocolo.py
```

7.8. Dashboard com fluxo serial simulado

O `tools/sim_serial_gateway.py` emula a saída serial do gateway (as mesmas linhas JSON do firmware, com codificação real dos quadros e canal sintético) num servidor TCP preso ao loopback, e o `tools/gateway_dashboard.py` lê esse fluxo com o mesmo código que lê uma porta COM.

1. No primeiro terminal, inicie o gateway simulado (3 drones, 600 s simulados, 10 vezes acelerados); ele imprime `aguardando conexao`:

```
python tools/sim_serial_gateway.py --port 8195 --speed 10 --duration 600 --drones 3
```

2. No segundo terminal, com o mesmo ambiente Python, capture por 75 s. A pasta recebe `brutos.jsonl`, `pacotes.csv`, `eventos.csv`, `resumo.json` e `dashboard.png`; a segunda linha reprocessa a captura versionada, para conferência imediata:

```
python tools/gateway_dashboard.py --port socket://127.0.0.1:8195 --duration 75 `
  --out ..\lora_saida\dash_sim "--gw-pos=-16.6799,-49.2550"
python tools/gateway_dashboard.py --replay resultados/dash_sim/brutos.jsonl `
  --out ..\lora_saida\replay
```

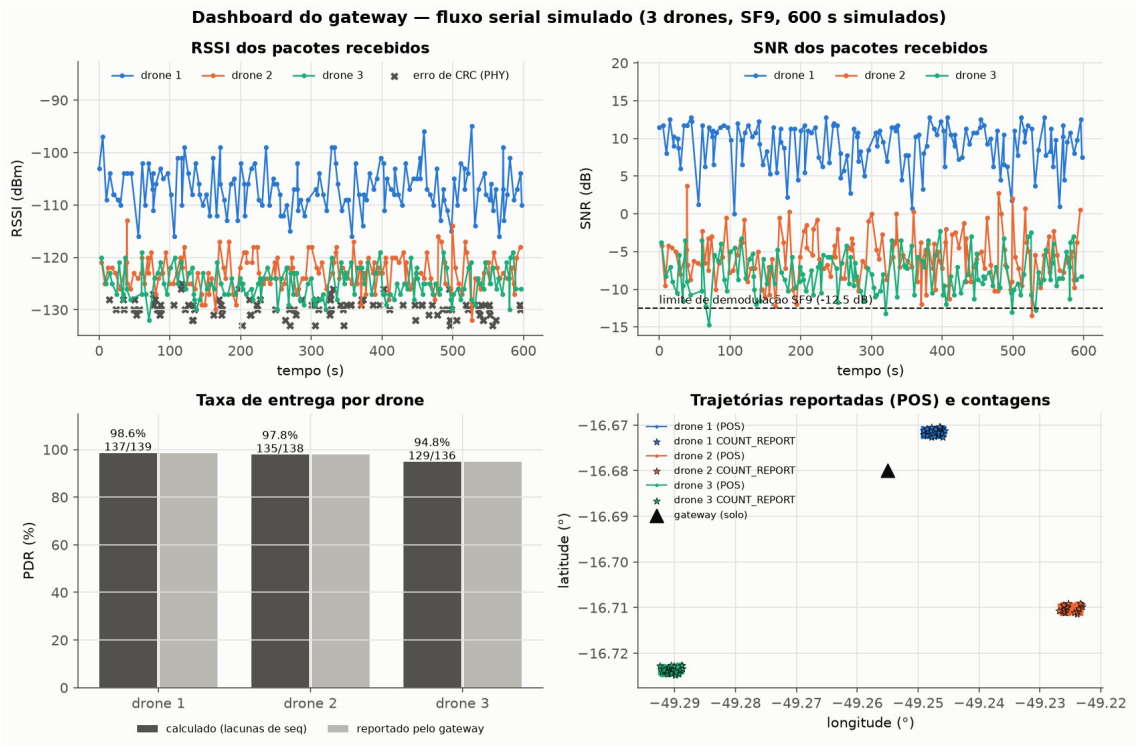


Figura 14. Painel esperado do dashboard com o fluxo simulado: RSSI, SNR, PDR por drone e trajetórias

7.9. Gabinete e documentos de hardware

1. Meça a placa com paquímetro: o gabinete supõe 50,2 × 25,5 mm (`--pcb-l`, `--pcb-w`) e 8,6 mm de componentes (`--comp-h`), valores não conferidos numa placa real. Gere base e tampa com as medidas obtidas:

```
python hardware/gabinete/gabinete.py --pcb-l 50.2 --pcb-w 25.5 --comp-h 8.6
```

As saídas são `hardware/gabinete/gabinete_base.stl`, `hardware/gabinete/gabinete_tampa.stl`, `resultados/gabinete.json` e `evidencias/fig_gabinete.png`. Imprima em PETG (cerca de 15 g de filamento) e fixe com parafusos M3x8 nas quatro orelhas (padrão de 30,5 mm). O repositório não define parâmetros de fatiamento; os 12,9 g estimados supõem preenchimento de 100 %.

2. Regere o diagrama de ligação e a estimativa de consumo (valores de datasheet, a substituir por medição):

```
python hardware/diagrama_ligacao.py
python hardware/orcamento_energia.py
```

7.10. Montagem da bancada

Esta etapa e as duas seguintes nunca foram executadas. Adquira os itens de `hardware/BOM.csv`, nas versões de 915 MHz.

Tabela 15. Lista de materiais da bancada (extrato de hardware/BOM.csv)

Qtd.	Item	Uso
2 (+1 reserva)	Heltec WiFi LoRa 32 V3, versão 863–928 MHz	nó e gateway
1	Antena omnidirecional 915 MHz, 3 dBi, SMA	gateway, em mastro de 2 a 3 m
1	Antena dipolo 915 MHz, 2 dBi, SMA	nó
2	Pigtail U.FL para SMA fêmea, 10 a 15 cm	placa até a antena
2	LiPo 1S 3,7 V, 1000 a 2000 mAh, JST 1,25 mm	nó sem USB
2	Cabo USB-C de dados	gravação e serial
1	GPS NEO-6M ou NEO-M8N, 3,3 V (opcional)	posição dos pacotes
1	Medidor de energia USB (opcional)	consumo real

Ligue conforme a parte (a) da figura:

1. Gateway: pigtail no U.FL, antena de 3 dBi vertical e USB-C ao notebook (alimentação e serial a 115200 bps).

2. Nó: dipolo de 2 dBi vertical e alimentação por USB-C ou LiPo 1S; confira a polaridade do JST antes de ligar.

3. GPS, só com `heltec_v3_node_gps`: TX do GPS no GPIO47, RX no GPIO48, 3V3 e GND. No `heltec_v3_node`, esses pinos são a UART1 (115200 8N1) do computador de bordo.

4. Nunca energize uma placa sem antena: o amplificador do SX1262 pode ser danificado.

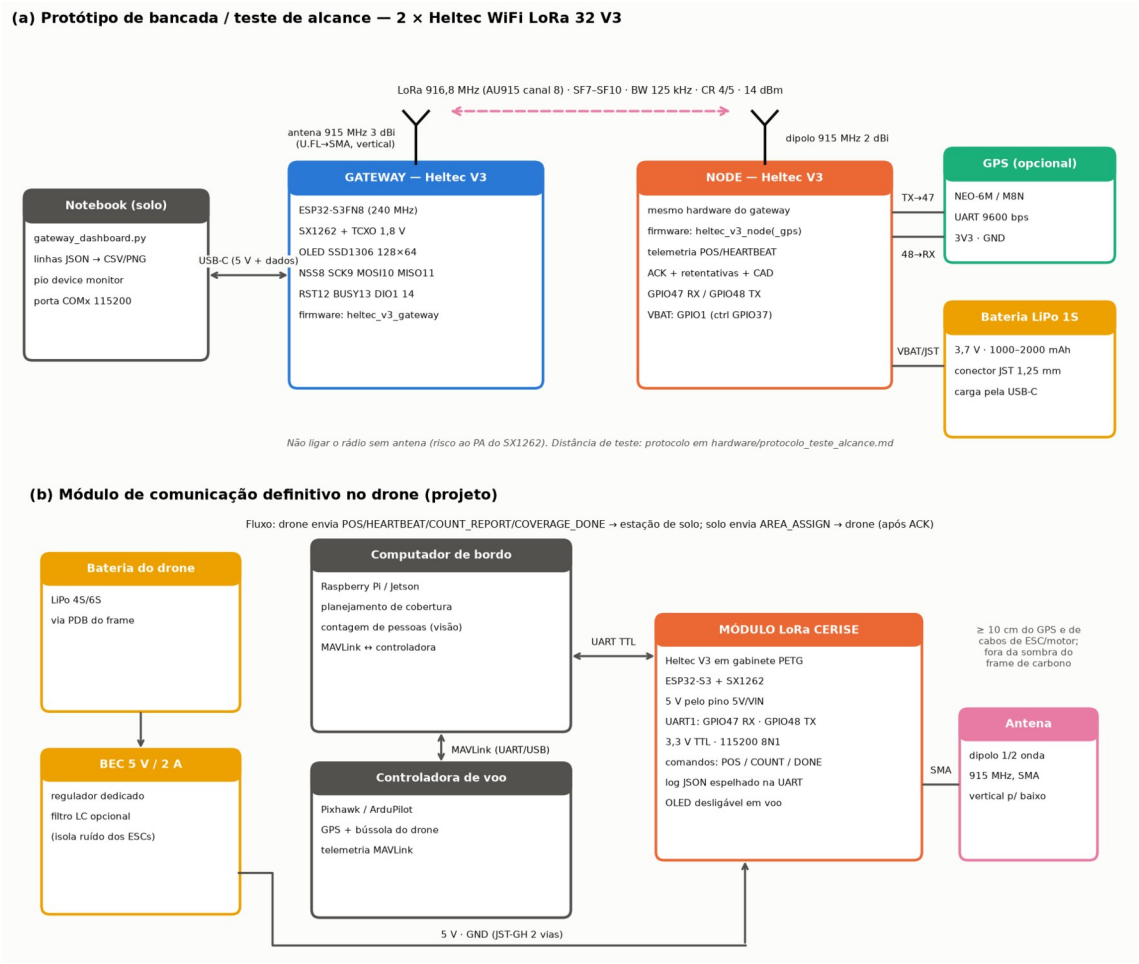


Figura 15. Diagrama de ligação: (a) bancada com duas placas Heltec V3 e notebook de solo; (b) módulo definitivo no drone

7.11. Gravação do firmware

1. Identifique as portas seriais (Gerenciador de Dispositivos; no Linux, `/dev/ttyUSB0` ou semelhante; as portas abaixo são exemplos) e, em `firmware/`, grave o gateway na placa 1 e o nó na placa 2:

```
pio run -e heltec_v3_gateway -t upload --upload-port COM6
pio run -e heltec_v3_node    -t upload --upload-port COM7
```

2. Abra o monitor do gateway: a linha `{"evt":"boot",...}` deve ter `"rc":0`, `"freq_mhz":916.8` e `"sf":9`. Digite `HELP` e `STATS` e saia com Ctrl+C; repita no nó (`-e heltec_v3_node --port COM7`) com `STATUS`:

```
pio device monitor -e heltec_v3_gateway --port COM6
```

3. Para gravar sem recompilar, siga `firmware/binarios/README.md`: use o esptool (pacote `esptool` do pip) com os endereços 0x0 (Heltec V3) ou 0x1000 (TTGO) para o bootloader, 0x8000, 0xe000 e 0x10000. O `boot_app0.bin` não é versionado; copie-o de `%USERPROFILE%\.platformio\packages\framework-arduinoespressif32\tools\partitions\` para `firmware/binarios/` e execute ali:

```
esptool.py --chip esp32s3 --port COM6 --baud 921600 write_flash -z `
  0x0 heltec_v3_gateway\bootloader.bin 0x8000 heltec_v3_gateway\partitions.bin `
  0xe000 boot_app0.bin 0x10000 heltec_v3_gateway\firmware.bin
```

7.12. Ensaio de alcance e PDR

O procedimento completo está em `hardware/protocolo_teste_alcance.md`. Faça primeiro as conferências de bancada, com as placas a cerca de 2 m e `POWER 2` nas duas:

1. Ambas iniciam com `"rc":0` e com as mesmas `freq_mhz` e `sf`.

2. O gateway imprime `POS` e `HEARTBEAT` com `"dup":false`, e o nó imprime `{"evt":"ack",...}`.

3. No gateway, `ASSIGN 1 3 7 -16.681 -49.2565 -16.679 -49.2535 40 25` faz o nó imprimir `{"evt":"dl",...}` e reportar a área 3.

4. Com a antena do gateway desconectada por 20 s (nunca a do transmissor ligado), o nó registra `timeout` e `drop`; ao reconectar, a entrega volta.

Tabela 16. Matriz do ensaio (hardware/protocolo_teste_alcance.md, seção 4)

Variável	Valores
Distância (visada direta)	50, 100, 200, 500, 1000, 2000 e 3000 m e o máximo alcançável
SF	7, 9 e 10 (SF11 e SF12 só como referência, com `DWELL 0`)
Potência	14 dBm; 20 dBm só no ponto mais distante
Quadro	POS (24 B); no ponto mais distante, também AREA_ASSIGN (32 B), só com SF ≤ 9
Altura do nó	1,5 m em tripé; com autorização, drone a 30, 60 e 100 m
Repetições	200 quadros por ponto, a cada 1 s

Em cada ponto:

1. Posicione o nó, anote coordenadas e distância e ajuste o mesmo SF nas duas placas pelo monitor (`SF 9`). Feche o monitor do gateway, pois a porta não pode ser aberta por dois programas.

2. Inicie a captura rotulada no notebook:

```
python tools/gateway_dashboard.py --port COM6 --label "d500_sf9_h1.5" `
  --out resultados/campo/d500_sf9
```

3. Num terminal serial ligado ao nó (por exemplo, outro notebook com `pio device monitor`), envie `RANGETEST 200 1000`: 200 quadros POS sem ACK nem retentativa, que medem a PDR bruta do enlace.

4. Após `{"evt":"rangetest_end",...}`, encerre o dashboard com Ctrl+C; o `resumo.json` traz PDR, RSSI e SNR médios e mínimos.

5. Repita por 5 min com a telemetria normal, com ACK, para medir a PDR após retentativas e a latência (`lat_ms` nos eventos `ack`); em SF11 e SF12, use só o `RANGETEST`, pois o ACK do gateway também passa de 400 ms (659,5 ms em

SF11). No ponto mais distante, repita com o AREA_ASSIGN (`ASSIGN` no gateway), só com SF ≤ 9.

6. Registre clima, obstáculos, orientação das antenas e altura na planilha da seção 7 do protocolo; para o consumo, siga a seção 6.

Regulamentação: o rádio opera em 915–928 MHz (Resolução ANATEL nº 680/2017, Ato nº 14.448/2017), plano AU915, canal 8, com no máximo 400 ms no ar por quadro: o POS cabe até SF10 (370,7 ms) e o AREA_ASSIGN até SF9 (246,8 ms). Só o nó aplica esse limite (evento `dwell_block`); o gateway não confere o tempo no ar dos *downlinks* (ACK e AREA_ASSIGN), por isso nunca envie `ASSIGN` com SF 10 ou maior. Use até 1 W e antenas de até 6 dBi, só em ensaios curtos; a operação definitiva exige salto de frequência e homologação. Voe só com drone cadastrado, em linha de visada, até 120 m (RBAC-E nº 94 e ICA 100-40).

7.13. Verificação do resultado

As etapas de software devem reproduzir a tabela abaixo; o dashboard simulado foi reexecutado e reproduziu exatamente os números de `resultados/dash_sim/resumo.json`.

Tabela 17. Valores de referência e arquivos de origem

Verificação	Valor esperado	Arquivo
Compilação	5 `SUCCESS`; `heltec_v3_node` com 22 400 B de RAM e 376 241 B de Flash	`resultados/build_all.log`
Testes nativos	30 testes, 0 falhas	`resultados/pio_test_native.txt`
Testes Python	21 aprovados (sem o repositório de drones, 20 e 1 pulado)	`resultados/pytest_tools.txt`
Tempo no ar do POS	61,7 / 205,8 / 370,7 ms em SF7 / SF9 / SF10	`resultados/airtime_mensagens.csv`
Alcance SF9, drone a 60 m	5,99 km (Hata suburbano), 3,00 km (urbano)	`resultados/enlace_alcance.csv`
PDR simulada, 10 drones, SF9	ALOHA 46,0 %; CSMA/CAD + ACK 62,6 %; TDMA + ACK 100 %	`resultados/sim_canal_vs_N.csv`
Simulador contra teoria	erro médio de 0,56 p.p.	`resultados/sim_canal_validacao_aloha.csv`
Dashboard simulado	593 linhas; PDR de 98,6 / 97,8 / 94,8 %	`resultados/dash_sim/resumo.json`
Gabinete	55,0 × 44,3 × 17,2 mm; 12,9 g	`resultados/gabinete.json`
Consumo estimado	0,23 a 0,34 W	`resultados/orcamento_energia.json`

Para o ensaio físico não há valores medidos: compare com `resultados/enlace_alcance.csv` e com os requisitos R4 (alcance de 3 km ou mais com margem de 10 dB) e R5 (PDR de 95 % ou mais) de `hardware/requisitos_modulo_drone.md`. Com 200 quadros, o intervalo de 95 % tem meia-largura de cerca de 4,2 p.p. para PDR de 0,9.

7.14. Problemas comuns e soluções

- **Testes nativos sem compilador no Windows:** instale o WinLibs, ponha `mingw64\bin` no PATH e abra um terminal novo.
- **Teste de compatibilidade pulado (SKIPPED):** defina `CERISE_DRONES_DIR` com a pasta do `cerise-2026-drones-segmentacao`.
- **Erro na posição do gateway:** o valor de `--gw-pos` começa com sinal de menos; use `"--gw-pos=-16.6799,-49.2550"`.
- **Dashboard simulado sem linhas:** inicie antes o `sim_serial_gateway.py`; ele aceita uma única conexão e termina ao fim da duração.
- **Porta serial ocupada:** monitor e dashboard não abrem a mesma porta ao mesmo tempo.
- **SF ou potência voltam ao padrão:** `SF` e `POWER` não são persistentes; se a linha `boot` reaparecer, reenvie-os nas duas placas ou altere `-DLORA_SF` e regrave.
- **Quadro bloqueado pelo limite de 400 ms:** o `dwell_block` só existe no nó e é esperado para o POS e os demais *uplinks* em SF11 e SF12. O gateway não bloqueia *downlinks*: não envie `ASSIGN` com SF 10 ou maior, pois o AREA_ASSIGN ocupa 452,6 ms em SF10.
- **OLED com "ERRO no radio":** o ambiente gravado não corresponde à placa (Heltec V3 com SX1262, TTGO com SX1276).
- **Tensão de bateria errada na Heltec V3.2:** `firmware/src/common/board.h` usa `VBAT_CTRL_ACTIVE LOW` (V3.0 e V3.1); na V3.2 é HIGH.
- **Ativação do Python bloqueada no PowerShell:** chame `. \.venv\Scripts\python.exe` diretamente, sem alterar a política de execução.

7.15. Referências do repositório

- `README.md`, `RELATORIO_SECAO.md` e `RELEASES.md`: visão geral, texto técnico e Release.
- `firmware/platformio.ini`, `firmware/lib/cerise_proto/src/cerise_proto.h`, `firmware/src/node_main.cpp` e `firmware/src/gateway_main.cpp`: ambientes, protocolo e comandos seriais.
- `firmware/test/test_proto/test_proto.c`, `tools/tests/` e `firmware/binarios/README.md`: testes e gravação.

- `hardware/BOM.csv`, `hardware/protocolo_teste_alcance.md`, `hardware/requisitos_modulo_drone.md` e `hardware/gabinete/gabinete.py`: hardware, ensaio e gabinete.
- `resultados/` e `evidencias/`: números de referência e figuras.

8. DRONES: SEGMENTAÇÃO DE ÁREA, DETECÇÃO E CONTAGEM DE PESSOAS E SIMULAÇÃO

8.1. O que será reproduzido

Este roteiro reproduz a frente de drones do Plano de Trabalho 2026: a divisão de uma área georreferenciada entre drones heterogêneos, com as missões de cobertura (Objetivo 5, Atividade 13); a realocação da área quando um drone abandona a missão (algoritmos de colaboração); e a detecção e contagem de pessoas em imagens aéreas (Objetivo 5, item 1), validada em vídeos reais e numa simulação 3D no NVIDIA Isaac Sim 5.1. O código está no repositório `cerise-2026-drones-segmentacao` da organização `cerise-ufg`.

Ao final você terá:

- o pacote `cerise_drones` com as missões de 4 drones em `.plan` (QGroundControl), KML e GeoJSON e o mapa em HTML;
- o benchmark dos métodos de partição e a simulação de falhas com realocação e custo LoRa;
- um YOLO11s ajustado para "pessoa" no VisDrone2019-DET, com mAP, erro de contagem e FPS;
- as métricas de rastreamento e de contagem única no VisDrone2019-MOT;
- o voo simulado de 4 drones sobre 11,3 ha do Campus Samambaia com 80 pessoas animadas, a contagem única global e o vídeo;
- o plano B Gazebo + PX4 pronto para construir (não executado no projeto).

8.2. Requisitos

Tabela 18. Máquinas usadas no projeto e alternativas

Etapa	Configuração usada	Alternativa
Segmentação, benchmark e colaboração	Windows 11 ou Linux, só CPU	qualquer computador com Python 3.11
Detecção, vídeos reais e contagem	NVIDIA DGX Spark (GPU GB10 Blackwell sm_121, CPU aarch64, 128 GB de memória unificada), Ubuntu 24.04, Docker + NVIDIA Container Toolkit	x86_64 com GPU NVIDIA e o mesmo Docker (não testado)
Simulação 3D	a mesma estação, `nvcr.io/nvidia/isaac-sim:5.1.0` (arm64)	x86_64 com GPU NVIDIA RTX; a imagem também é publicada para amd64 (não testado)

Software:

- Python 3.11 (testado com 3.11.9), com as bibliotecas nas versões do ambiente usado no projeto, fixadas no comando de instalação do passo 1 da segmentação (numpy 2.4.6, shapely 2.2.0, pyproj 3.7.2, scikit-learn 1.9.1, scipy 1.17.1, matplotlib 3.11.2, folium 0.20.0, imageio 2.38.1, pillow 13.0.0, playwright 1.64.1, pandas 3.1.0 e pytest 9.1.1);
- Docker com `--gpus all`, `sudo`, `git`, GitHub CLI (`gh`), `curl` e `ffmpeg` com libx264 no host;
- `nvcr.io/nvidia/pytorch:26.05-py3` (PyTorch 2.12) com ultralytics 8.4.189 (`deteccao/logs/build.log`); como o `deteccao/Dockerfile` instala a versão mais recente, troque `ultralytics` por `ultralytics==8.4.189` para fixá-la;
- Isaac Sim 5.1.0, que exige driver NVIDIA 570.169 ou mais novo (o projeto usou o 580.178.04).

Acessos: leitura na organização `cerise-ufg` do GitHub (repositórios privados; solicite ao bolsista); internet para o VisDrone, os pesos COCO e os ativos do Isaac; e o aceite, por você mesmo, da *NVIDIA Isaac Sim Additional Software and Materials License* (https://www.nvidia.com/en-us/agreements/enterprise-software/isaac-sim-additional-software-and-materials-license/).

Disco: cerca de 100 GB (imagem do Isaac 30 GB, imagens próprias dezenas de GB, VisDrone-DET 4,8 GB, VisDrone-MOT 7,4 GB, Releases 2,96 GB, 0,8 GB por voo simulado). Tempos medidos: benchmark 584 s, download do VisDrone-DET 10 min, treino 48,7 min, detecção e rastreamento nos vídeos 11 min, download da imagem do Isaac 22,5 min, render de cada voo 21 min. Conte de um a dois dias.

8.3. Obtenção do código e dos dados

1. Clone o repositório. Os scripts de Docker usam a pasta fixa `~/cerise2026/drones` (montada como `/w`) e chamam a pasta da simulação de `sim`; clone nesse caminho e crie o link relativo, que vale dentro e fora dos contêineres. O link e o que os scripts criam (como `logs/`, `home/` e `pylib_mot/`) aparecem como não rastreados no `git status`; não os versione.

```
gh auth login
mkdir -p ~/cerise2026/hf_cache
gh repo clone cerise-ufg/cerise-2026-drones-segmentacao ~/cerise2026/drones
cd ~/cerise2026/drones && ln -s simulacao sim
```

2. Baixe os Releases que quiser (nenhum é obrigatório). O tag publicado é `dados-2026-10-26`. `sim_saidas_run01.tar` e `sim_saidas_run02a30.tar` trazem os quadros dos voos renderizados, `validacao_videos_cache_det.tar` as detecções dos vídeos e `deteccao_runs.tar` as `runs/` do treino. Use `-p <arquivo>` para baixar só alguns; extraia na raiz do repositório:

```
gh release download dados-2026-10-26 -R cerise-ufg/cerise-2026-drones-segmentacao \
  -D ~/cerise2026/_releases
cd ~/cerise2026/_releases && sha256sum -c --ignore-missing SHA256SUMS.txt
cd ~/cerise2026/drones && for f in ~/cerise2026/_releases/*.tar; do tar xf "$f"; done
```

3. O peso final do detector já está no git (`resultados/deteccao/best_yolo11s_visdrone_pessoa_960.pt`). O VisDrone (dados de terceiros, uso acadêmico) e os ativos do Isaac Sim (licença da NVIDIA) não são redistribuídos: você os baixa nas etapas a seguir.

8.4. Segmentação da área e geração das missões

1. Crie o ambiente e rode os testes (no Windows, use `.venv\Scripts\python`). O pacote não tem instalador; os scripts põem a raiz do repositório no `sys.path`.

```
cd ~/cerise2026/drones
python3.11 -m venv .venv
.venv/bin/python -m pip install numpy==2.4.6 shapely==2.2.0 pyproj==3.7.2 \
  scikit-learn==1.9.1 scipy==1.17.1 matplotlib==3.11.2 folium==0.20.0 \
  imageio==2.38.1 pillow==13.0.0 playwright==1.64.1 pandas==3.1.0 pytest==9.1.1
.venv/bin/python -m playwright install chromium
.venv/bin/python -m pytest -q tests
```

2. Rode a demonstração. Ela divide o campo aberto do Campus Samambaia (`dados/ufg_samambaia_campo_aberto.geojson`, 34,23 ha, OpenStreetMap) entre 4 drones dos tipos A, B, C e A (`cerise_drones/frota.py`) por faixas, k-means, DARP e DARP + refinamento, e exporta o plano final em `resultados/missao_ufg/` (`.plan`, `.kml` e `.geojson` por drone e `mapa_ufg_darp_ref.html`), além das figuras `evidencias/01` a `03`:

```
.venv/bin/python scripts/demo_ufg.py
```

3. Para uma área sua, crie `minha_missao.py` na raiz. Zonas proibidas entram em `AreaMissao.de_arquivos(nome, area, [zonas],`

`margem_zona_m=10`); bases em lon/lat são convertidas com `area.proj.ponto_xy(lon, lat)`. Com o polígono da UFG, o exemplo imprime `makespan 6.57 min  CV 0.002  cobertura 100.00 %` e grava 13 arquivos e o mapa em `saida_missao/`:

```
from cerise_drones import AreaMissao, criar_frota, planejar_missao
from cerise_drones.cenarios import bases_no_contorno
from cerise_drones.visualizar import mapa_folium

area = AreaMissao.de_arquivos("minha area",
                              "dados/ufg_samambaia_campo_aberto.geojson")
frota = criar_frota(["A", "B", "C", "A"], bases_no_contorno(area, 4), altitude_m=40)
plano = planejar_missao(area, frota, "darp", refinar=4, sementes="centros")
m = plano.metricas
print(f"makespan {m['makespan_s'] / 60:.2f} min  CV {m['cv_tempo']:.3f}  "
      f"cobertura {m['cobertura_pct']:.2f} %")
plano.exportar("saida_missao", "missao")
mapa_folium(plano, "saida_missao/mapa.html", "Minha missao")
```

4. Rode o benchmark (4 cenários, 2 a 6 drones, frotas homogênea e heterogênea), a ablação, os gráficos, as falhas com realocação (repartição e leilão) e a análise LoRa. Eles regravam `resultados/` e `evidencias/`; compare com `git diff --stat`.

```
.venv/bin/python scripts/benchmark.py
.venv/bin/python scripts/benchmark.py --ablacao
.venv/bin/python scripts/graficos_benchmark.py
.venv/bin/python scripts/colaboracao_sim.py
.venv/bin/python scripts/protocolo_analise.py
```

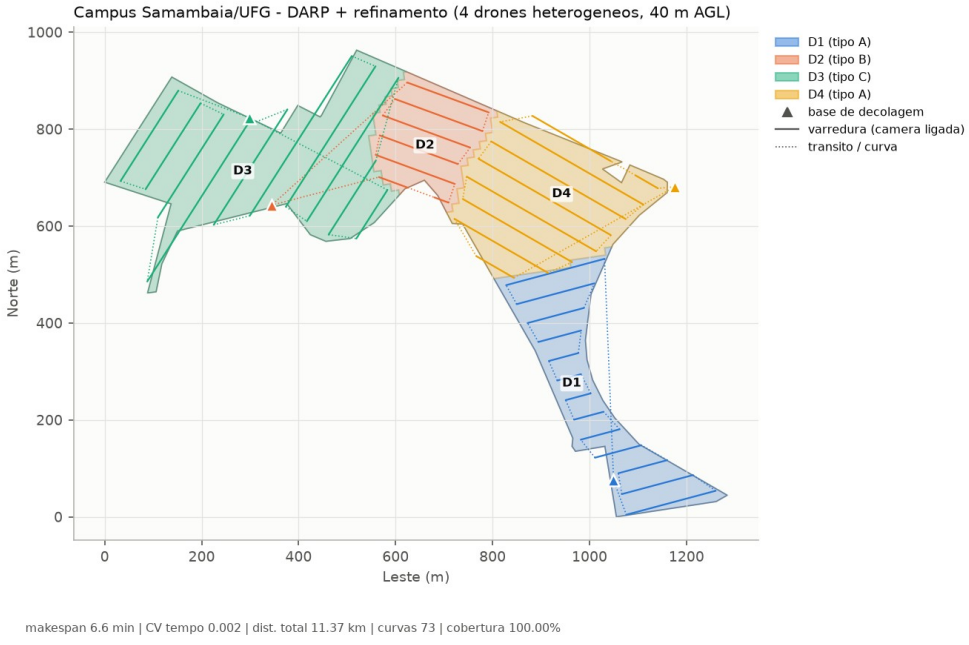


Figura 16. Plano esperado: DARP + refinamento no Campus Samambaia com 4 drones heterogêneos a 40 m, com subáreas, rotas e bases

8.5. Detector de pessoas YOLO11s no VisDrone2019-DET

Rode na estação com GPU, em `~/cerise2026/drones`. O `deteccao/run.sh` sobe o contêiner `cerise26_drones_<nome>` em segundo plano, com log em `logs/<nome>.log`; espere cada passo com `sudo docker wait`.

1. Construa a imagem `cerise26_drones:latest`. Em x86_64 o Dockerfile deve servir, pois a imagem NGC também existe para essa arquitetura, mas isso não foi testado.

```
bash deteccao/build.sh
```

2. Baixe os zips oficiais (URLs do `VisDrone.yaml` do ultralytics) e prepare a classe única "pessoa" (pedestrian + people), com o treino reduzido a 960 px:

```
NP=12 bash deteccao/00_fetch_visdrone_parallel.sh
bash deteccao/run.sh prep 02_prep_pessoa.py
sudo docker wait cerise26_drones_prep && tail -3 logs/prep.log
```

3. Teste a precisão numérica. No GB10 o FP16 gerou NaN; por isso os scripts usam FP32 (`amp=False`, `half=False`). Em outra GPU, rode este teste antes de ativar FP16:

```
bash deteccao/run.sh amp teste_amp.py
```

4. Treine. O treino é limitado pelo relógio (`--horas`) e por um teto de memória de GPU (`--teto_gb`, padrão 7,5 GB). O projeto fez 5 épocas em 48,7 min, com pico de 6,5 GB; numa GPU mais lenta, aumente `--horas`. A opção `--teste` faz 1 época em 5 % do treino; `--workers 8` repete a execução registrada em `deteccao/logs/treino.log` (o padrão do script é 6).

```
bash deteccao/run.sh treino 04_treinar.py --modelo yolo11s.pt --imgsz 960 \
  --batch 4 --horas 0.8 --workers 8
sudo docker wait cerise26_drones_treino
```

5. Se pulou o treino, copie o peso versionado para o caminho que a avaliação e as etapas seguintes esperam:

```
mkdir -p runs/ft_pessoa_yolo11s_960/weights
cp resultados/deteccao/best_yolo11s_visdrone_pessoa_960.pt \
  runs/ft_pessoa_yolo11s_960/weights/best.pt
```

6. Avalie o YOLO11s COCO e o ajustado (mAP no val e no test-dev, erro de contagem e FPS). A avaliação exige o peso do passo 4 ou do passo 5: sem ele, o `06_avaliar_tudo.py` avalia só o modelo COCO, sem aviso. A saída vai para `resultados/avaliacao_deteccao.json`; a de referência está em `resultados/deteccao/`.

```
bash deteccao/run.sh avaliacao 06_avaliar_tudo.py
```



Figura 17. Exemplos esperados do detector ajustado no VisDrone2019-DET, com a contagem real e a detectada no limiar 0,175

8.6. Contagem em vídeos reais (VisDrone2019-MOT)

1. Baixe o val e o test-dev pelos links oficiais do Google Drive (o script retoma o download e espera quando o Drive responde *Quota exceeded*; os SHA-256 estão na seção 2 de `validacao_videos/RELATORIO_SECAO_VIDEOS.md`) e instale `lap`, `motmetrics` e `xmltodict` em `pylib_mot/`:

```
cd ~/cerise2026/drones/validacao_videos/scripts
nice -n 19 python3 -I 00_baixar_visdrone_mot.py
bash 01_preparar_libs.sh
```

2. Rode o pipeline: `10_detectar.py` guarda em cache as detecções de 4 variantes (ajustado e COCO, 960 e 1280 px, FP32); `20_gmc_condicoes.py` calcula a compensação de movimento de câmera (GMC); `30_rastrear.py` escolhe os parâmetros do ByteTrack e do BoT-SORT numa grade no val e os aplica sem alteração ao test-dev; `32_ablacao_gmc.py` tira a GMC; `40_fps.py` mede a vazão. O peso ajustado deve estar em `runs/ft_pessoa_yolo11s_960/weights/best.pt`; o `weights/yolo11s.pt` é baixado pelo ultralytics.

```
CPUS=6 bash run_vv.sh det 10_detectar.py
sudo docker wait cerise26_drones_vv_det
GPU=0 CPUS=4 bash run_vv.sh gmc 20_gmc_condicoes.py
sudo docker wait cerise26_drones_vv_gmc
GPU=0 CPUS=4 bash run_vv.sh rastreio 30_rastrear.py
sudo docker wait cerise26_drones_vv_rastreio
GPU=0 CPUS=4 bash run_vv.sh gmcabl 32_ablacao_gmc.py
sudo docker wait cerise26_drones_vv_gmcabl
CPUS=4 bash run_vv.sh fps 40_fps.py
```

Os logs ficam em `~/cerise2026/drones/logs/vv_<nome>.log` e os resultados regravam `validacao_videos/resultados/`. Com o Release `validacao_videos_cache_det.tar`, pule o `10_detectar.py`; a GMC já está versionada em `validacao_videos/cache_gmc/`.

8.7. Simulação 3D no Isaac Sim 5.1

1. Leia e aceite a licença do Isaac Sim (ver Requisitos); só então passe `ACCEPT_EULA=Y`. `PRIVACY_CONSENT=N` desliga a telemetria.

2. Baixe a imagem oficial (não exigiu login no NGC), construa a imagem da contagem e rode os testes do pacote `cerise_sim`:

```
sudo docker pull nvcr.io/nvidia/isaac-sim:5.1.0
cd ~/cerise2026/drones/sim
sudo nice -n 19 docker build -f Dockerfile.contagem -t cerise26_drones_sim:latest .
bash scripts/rodar_testes_spark.sh
```

3. Faça o espelho local dos ativos oficiais (personagens, animações, `Biped_Setup.usd`, esqueleto `biped_demo` e quadricóptero); `--listar` só mostra os tamanhos:

```
python3 isaac/baixar_ativos.py --destino ~/cerise2026/drones/sim/isaac_ativos --listar
python3 isaac/baixar_ativos.py --destino ~/cerise2026/drones/sim/isaac_ativos
```

4. Use os cenários versionados `cenario/reduzido` (recorte de 11,27 ha, 4 drones a 40 m nas rotas DARP + refinamento, 80 pessoas, 173 s) e `cenario/reduzido_a30` (o mesmo a 30 m). Para criar outro, use `scripts/gerar_cenario.py` com o ambiente da segmentação; este comando refaz o `reduzido_a30` com outro nome:

```
../.venv/bin/python scripts/gerar_cenario.py --nome meu_a30 \
  --recorte=-300,-90,110,360 --altitude 30
```

5. Rode o teste de fumaça (1 drone, 6 pessoas, 4 s; a primeira execução, com a compilação de shaders, levou cerca de 2 min). O `isaac/rodar_isaac.sh` exige `ACCEPT_EULA=Y` e 30 GB de memória *available* (`MIN_LIVRE_INICIO_GB`), limita o contêiner a 24 GB (`MEM_LIMITE`) e 8 CPUs (`CPUS`) e liga o vigia `isaac/vigia_memoria.sh`, que mata o contêiner abaixo de 12 GB *available* (`MIN_LIVRE_GB`). Ele também recusa iniciar se houver um contêiner `braco_gen` (outro trabalho do projeto), o que não deve ocorrer na sua máquina.

```
export ACCEPT_EULA=Y PRIVACY_CONSENT=N
export ATIVOS_LOCAIS=$HOME/cerise2026/drones/sim/isaac_ativos
bash isaac/rodar_isaac.sh meu_smoke --smoke
sudo docker logs -f cerise26_isaac_meu_smoke
```

6. Renderize os dois voos com nomes novos: `saidas/run01` e `saidas/run02a30` guardam os metadados do projeto, e o script passa a pasta de saída para o usuário do contêiner (uid 1234).

```
bash isaac/rodar_isaac.sh meu_run01 --cenario /cenario/reduzido --fps 5
sudo docker wait cerise26_isaac_meu_run01
bash isaac/rodar_isaac.sh meu_run02a30 --cenario /cenario/reduzido_a30 --fps 5 \
  --sem_visao_geral
```

7. Conte as pessoas. O `scripts/contar.py` detecta com o YOLO11s ajustado (FP32, 1280 px), rastreia com cinco variantes (BoT-SORT recomendado pelos vídeos reais, BoT-SORT calibrado `botsort_cal`, rastreador próprio no solo, BoT-SORT com GMC pela pose e sem GMC), projeta as trilhas

no solo e deduplica pela posição, não pelos IDs. Para 30 m, troque `meu_run01` por `meu_run02a30` e o cenário por `cenario/reduzido_a30`:

```
bash scripts/rodar_contagem_spark.sh cont_meu_run01 scripts/contar.py \
  --execucao saidas/meu_run01 --cenario cenario/reduzido --detector yolo \
  --pesos /w/resultados/deteccao/best_yolo11s_visdrone_pessoa_960.pt --imgsz 1280 \
  --rastreadores botsort,botsort_cal,chao,botsort_pose,botsort_semgmc \
  --saida resultados/contagem_meu_run01.json --figura
sudo docker wait cerise26_drones_cont_meu_run01 && grep '^\[' logs/cont_meu_run01.log
```

Sem renderizar, reproduza a contagem do projeto com o cache versionado: repita o comando com `--execucao saidas/run01`, `--cache_det resultados/contagem_run01_cache_det.npz` e `--saida resultados/repro_contagem_run01.json`.

8. Gere o vídeo (visão geral, quatro câmeras com gabarito em verde e trilhas em vermelho, contagem acumulada) e monte o MP4 no host:

```
bash scripts/rodar_contagem_spark.sh video_meu_run01 scripts/video_relatorio.py \
  --execucao saidas/meu_run01 --contagem resultados/contagem_meu_run01.json \
  --deteccoes resultados/contagem_meu_run01_deteccoes_botsort_cal.csv \
  --quadros_dir tmp/video_meu_run01
sudo docker wait cerise26_drones_video_meu_run01
bash scripts/montar_video.sh tmp/video_meu_run01 evidencias/meu_video_run01.mp4 15
```

Em x86_64 com GPU dedicada (não testado), as verificações do script medem só a RAM do host; o uso de memória de vídeo não foi medido.



Figura 18. Cena esperada no Isaac Sim (t = 103,4 s): visão geral com os quatro drones e câmera do D4 em nádir a 40 m com as caixas de gabarito do Replicator

8.8. Plano B: Gazebo Harmonic + PX4 SITL

O plano B foi preparado, mas não foi construído nem executado; trate-o como não testado. O `simulacao/gazebo/Dockerfile` parte do `ubuntu:22.04`, instala o Gazebo Harmonic e compila o PX4 v1.15.4 (`px4_sitl_default`); como usa `dpkg --print-architecture`, deve servir também em x86_64. O `rodar_gazebo.sh` regrava o mundo versionado `gazebo/mundo_reduzido/` com o `gerar_mundo.py` (imagem `cerise26_drones`) e sobe o `gz sim` sem tela, uma instância PX4 por drone, a captura das câmeras (`capturar.py`) e as missões `.plan` via MAVSDK (`rodar_missoes.py`):

```
cd ~/cerise2026/drones/sim/gazebo && mkdir -p ../logs
sudo nice -n 19 docker build --build-arg JOBS=4 -t cerise26_gzpx4:latest . \
  > ../logs/build_gzpx4.log 2>&1
bash rodar_gazebo.sh gz_run01 reduzido 5
```

A saída tem o formato da do Isaac; conte com o passo 7 acima e `--execucao saidas/gz_run01`. As estimativas da seção 5 de `simulacao/PRONTO.md` não foram medidas.

8.9. Verificação do resultado

Tabela 19. Resultados de referência medidos no projeto

Etapa	O que deve aparecer	Arquivo de referência
Testes	`16 passed` (pacote) e `17 passed` (simulação)	`resultados/pytest_log.txt`, `simulacao/aux_execucao/pytest_final.txt`
Demonstração UFG	DARP + refinamento: makespan 6,57 min, CV 0,002, 11,37 km, cobertura 100 %; 13 arquivos exportados	`resultados/demo_ufg_log.txt`, `resultados/demo_ufg_metricas.json`
Benchmark	DARP + refinamento a 0,7-0,8 % do melhor makespan em média, 31 vitórias em 40 casos	`resultados/benchmark_resumo.md`
Colaboração	sem realocação perdem-se 9,2 % da área em média; a repartição recupera a cobertura com +23,1 % de makespan	`resultados/colaboracao.csv`
Detector	mAP50 0,528 no val (COCO 0,384) e 0,316 no test-dev; MAE de contagem 6,91 (val) e 8,76 (test-dev); 25,5 FPS a 960 px	`resultados/deteccao/avaliacao_deteccao.json`
Vídeos reais	ajustado 960 + BoT-SORT no test-dev: HOTA 0,328, IDF1 0,399, MAE 7,46; 2.095 IDs para 1.012 pessoas	`validacao_videos/resultados/metricas_agregadas.json`
Isaac Sim	`navmesh assada: True`; voo a 40 m com 866 quadros a 1,46 s/quadro e 11,5 GB no contêiner	em `simulacao/`: `resultados/logs/isaac_smoke6.log`, `resultados/logs/isaac_run01.log` e `saidas/run01.vigia.log`
Contagem simulada	AP50 0,702 (40 m) e 0,652 (30 m); `botsort_cal`: contagem única de 78 para 79 pessoas vistas (40 m; 13 duplicatas e 14 perdidas) e de 70 para 73 (30 m; 16 duplicatas e 19 perdidas)	em `simulacao/resultados/`: `logs/cont_run01.log`, `logs/cont_run02a30.log`, `contagem_run01.json` e `contagem_run02a30.json`

Com o cache versionado, a contagem deve repetir os números do log. Num render novo, compare a ordem de grandeza: a reprodutibilidade bit a bit do render não foi verificada.

8.10. Problemas comuns e soluções

- **NaN em FP16 no GB10.** Com o PyTorch 2.12 do NGC 26.05, o FP16 gerou NaN e 263 caixas onde o FP32 achava 14 (`deteccao/logs/amp.log`). Mantenha `amp=False` e `half=False`.

- **Contagem inflada.** Não conte pelos IDs do rastreador: nos vídeos foram 2.095 IDs para 1.012 pessoas e, na simulação, a soma de detecções deu 1.306 para 79 pessoas. Use a deduplicação no solo do `contar.py`.
- **Subcontagem no simulado.** O BoT-SORT recomendado pelos vídeos contou 34 de 79, porque o `fuse_score` multiplica a IoU por pontuações baixas; use o `botsort_cal`. Sem GMC (`botsort_semgmc`), nada é rastreado a 5 fps.
- **Falta de memória.** A navmesh do cenário `completo` (1.330 × 998 m) consumiu mais de 34 GB em cerca de 22 s e causou OOM global (`simulacao/resultados/logs/oom_run03completo_dmesg.txt`). Use o cenário reduzido e mantenha `MEM_LIMITE` e o vigia.
- **Personagens em pose fixa e gabarito em (0, 0).** Com o aviso `Type mismatch for variable Action/Walk`, falta o esqueleto `biped_demo`; refaça o espelho com o `baixar_ativos.py` atual, que já o inclui.
- **Pessoas andando em linha reta em direções arbitrárias.** A navmesh ficou pequena; confira no `log_sim.txt` a linha `navmesh assada: True` com volume de cerca de 452 × 467 m.
- **Peso não encontrado.** A avaliação e os vídeos esperam `runs/ft_pessoa_yolo11s_960/weights/best.pt`; copie o peso versionado (passo 5 da detecção).

8.11. Referências do repositório

- `README.md`, `RELEASES.md` e `RELATORIO_SECAO.md`: visão geral, artefatos grandes e métodos;
- `cerise_drones/` (`geo.py`, `frota.py`, `particao.py`, `cobertura.py`, `planejador.py`, `exportar.py`, `colaboracao.py`, `protocolo.py`) e `scripts/`;
- `deteccao/Dockerfile`, `deteccao/run.sh`, `deteccao/04_treinar.py`, `deteccao/05_avaliar_ft.py`;
- `validacao_videos/README.md` e `validacao_videos/RELATORIO_SECAO_VIDEOS.md`;
- `simulacao/PRONTO.md` e `simulacao/RELATORIO_SECAO_SIMULACAO.md`: preparo, correções de API e resultados;
- `simulacao/isaac/sim_ufg_isaac.py`, `simulacao/isaac/rodar_isaac.sh`, `simulacao/isaac/vigia_memoria.sh`, `simulacao/scripts/contar.py`, `simulacao/cerise_sim/botsort_chao.py` e `simulacao/gazebo/`.

9. CONTATO

Os pedidos de acesso aos repositórios e as dúvidas sobre a reprodução devem ser dirigidos ao bolsista, responsável pelo código e administrador da organização cerise-ufg no GitHub. Ao relatar um problema, informe a seção e o passo deste roteiro, o comando executado, a saída obtida (ou o arquivo de log) e a máquina usada; com acesso concedido, também é possível abrir uma *issue* no repositório da frente.

- Bolsista: Eduardo Martins de Carvalho Caixeta, CERISE/EMC/UFG.
- Coordenador dos eixos de Navegação Autônoma e Gêmeos Digitais: Prof. Dr. Alisson Assis Cardoso.
- Coordenador do eixo de Realidade Aumentada: Prof. Dr. Carlos Galvão Pinheiro Junior.